

석사학위 논문

효율적인 OSS 유사도 검사를 위한
PDG 기반의 소스코드 유사도 검사 및
 k -walk 생성에 관한 연구

A Study on PDG-based Source Code Similarity Checking
and k -walk Generation for Efficient OSS Similarity
Checking

2025년 2월

순천향대학교 대학원
소프트웨어융합학과
윤 성 철

효율적인 OSS 유사도 검사를 위한
PDG 기반의 소스코드 유사도 검사 및
 k -walk 생성에 관한 연구

A Study on PDG-based Source Code Similarity Checking
and k -walk Generation for Efficient OSS Similarity
Checking

지도교수 김 수 현

이 논문을 석사학위 논문으로 제출함

2024년 11월

순천향대학교 대학원
소프트웨어융합학과
윤 성 철

윤 성 철의 공학 석사학위논문을 인준함

2024년 11월 11일

위 원 장 홍 민 (인)

위 원 이 임 영 (인)

위 원 김 수 현 (인)

순천향대학교 대학원

목 차

제 1장 서론	1
1.1 연구 배경	1
1.2 연구 목표	5
제 2장 관련 연구	7
2.1 그래프 기반의 소스코드 유사도 검사 알고리즘	7
2.2 PDG	11
2.3 그래프 편집 거리	13
2.4 휴리스틱 알고리즘	14
2.5 Tottering 현상	18
제 3장 기존방식 분석	20
3.1 그래프 기반의 소스코드 유사도 검사 알고리즘	20
3.2 k -walk 기반의 휴리스틱 알고리즘	34
제 4장 요구사항 분석	37
4.1 그래프 기반의 소스코드 유사도 검사 알고리즘	37
4.2 k -walk 기반의 휴리스틱 알고리즘	39

제 5장 제안방식	42
5.1 Tottering 현상을 방지하는 k -walk 생성 기법	43
5.2 PDG 기반의 소스코드 유사도 검사 기법	50
제 6장 제안방식 분석	56
6.1 Tottering 현상을 방지하는 k -walk 생성 기법	56
6.2 PDG 기반의 소스코드 유사도 검사 기법	60
제 7장 결 론	65
참고 문헌	66
ABSTRACT	73

표 목차

〈표 I-1-1〉 소스코드 유사도 검사 기법 비교분석표	4
〈표 II-1-1〉 그래프 기반의 소스코드 유사도 검사 알고리즘 비교분석표	7
〈표 II-3-1〉 그래프 유사도 검사 기법 비교분석표	15

그림 목차

<그림 I-1-1> 국내 기업 OSS 활용률	1
<그림 I-1-2> 국내 OSS 시장 규모	2
<그림 II-1-1-1> 기호적 표현으로 변환된 소스코드 예시	8
<그림 II-1-2-1> 추상 구문 트리 예시	10
<그림 II-1-3-1> 소스코드 흐름도 예시	11
<그림 II-2-1> Program Dependence Graph 예시	12
<그림 II-5-2> 사이클 구조 예시	19
<그림 III-1-1> Luo et al. CoP 구조	21
<그림 III-1-2> 경로 유사도 검사 예시	24
<그림 III-1-3> Suttichaya et al. 알고리즘 구조	26
<그림 IV-2-1-1> dot 파일 예시	39
<그림 IV-2-2-1> MOSS 동일 소스코드 비교 결과	40
<그림 IV-2-3-1> MOSS 변형 소스코드 비교 결과	41
<그림 V-1> 제안방식 목표	42
<그림 V-1-1> 제안방식 초기화 단계 예시	44
<그림 V-1-2> 제안방식 탐색 단계 예시	45
<그림 V-1-3> 제안방식 탐색 단계 예시(Tottering 미발생)	47
<그림 V-1-4> 제안방식 탐색 단계 예시(Tottering 발생)	48
<그림 V-2-1> PDG 기반의 소스코드 유사도 검사 기법 시나리오	50
<그림 VI-1-1> k-walk 생성 시 사용한 그래프	56
<그림 VI-1-1-1> k-walk 생성 수	57

<그림 VI-1-2-1> k-walk 생성 시간	58
<그림 VI-1-3-1> 근사값 결과	60
<그림 VI-2-2-1> 동일 소스코드 기반 유사도 검사 결과	62
<그림 VI-2-3-1> 동일 소스코드 기반 유사도 검사 결과	63
<그림 VI-2-3-2> 동일 소스코드 기반 유사도 검사 결과	64

국 문 요 약

현 개발 환경은 OSS를 적극적으로 활용하여 개발 시간 및 비용을 단축한다. OSS를 이용 시, 개발자는 OSS의 사용 규정인 OSS 라이선스를 준수해야 한다. 하지만 OSS 라이선스는 라이선스마다 규정이 달라 상호 호환되지 않을 수 있다. OSS는 저작권의 보호를 받는 엄연한 지적 재산이므로, OSS 라이선스 위반 시 저작권 위반 등으로 처벌을 받을 수 있다. 그러나 여러 개발자가 협업하는 개발 환경의 특성상 다른 개발자가 사용한 모든 OSS를 식별하는 것은 힘들며, 이와 혼용할 수 없는 OSS임을 개발자가 육안으로 확인하는 것은 어려운 일이다. 이로 인해 개발이 완료된 상업적 소프트웨어를 판매 중지하거나 전체 소스코드를 공개해야 하는 등의 문제가 발생할 수 있다. 본 논문에서는 이러한 OSS 혼용 문제를 방지하고자 소프트웨어 내 OSS를 식별하기 위한 새로운 소스코드 유사도 검사 기법을 제안한다. 본 제안방식은 소스코드를 그래프로 변환하여 소스코드의 구조적 유사도를 비교하여 유사도를 측정한다. 이 경우, 기존 텍스트 기반의 소스코드 유사도 검사에서 발생한 소스코드 구문 변경 및 구조 변경 등의 유사도 탐지 회피를 효과적으로 방지하여 더욱 정확한 유사도 검사가 가능하다. 하지만 그래프로 변환 후 소스코드 유사도 검사 시 추가적인 계산 비용이 발생해 효율성이 저하된다. 이를 해결하기 위해 기존 k -walk 기반의 그래프 휴리스틱 알고리즘 중 k -walk 생성 과정에서 발생했던 Tottering 현상을 방지하는 새로운 k -walk 생성 알고리즘을 제안한다. 이는 더욱 정확하고 효율적으로 소프트웨어 내 OSS를 식별할 수 있어 OSS 식별이 필요한 개발 환경, 소스코드 유사도 검사가 필요한 교육 환경 등에 활발히 사용될 것으로 기대된다.

키워드

Open Source Software, 소스코드 유사도, 그래프, 프로그램 종속성 그래프, 그래프 유사도, 휴리스틱 알고리즘

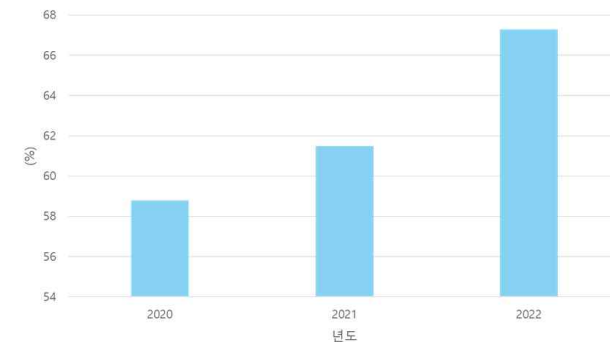
제 1장 서론

본 장에서는 본 제안방식의 연구에 대한 배경 및 목표를 기술한다. 1.1절 연구 배경을 통해 본 연구에 대한 동기, 현재 연구 분야에서의 문제점, 본 연구의 필요성 및 중요성을 설명하고, 1.2절 연구 목표를 통해 본 연구의 목표 및 제안방식의 개념을 설명한다. 또한, 해당 제안방식을 통해 달성하고자 하는 기대효과와 더불어, 기존 연구와의 차별성 및 본 연구를 통해 해결하고자 하는 기존 연구의 문제점에 대해 간단히 설명한다.

1.1 연구 배경

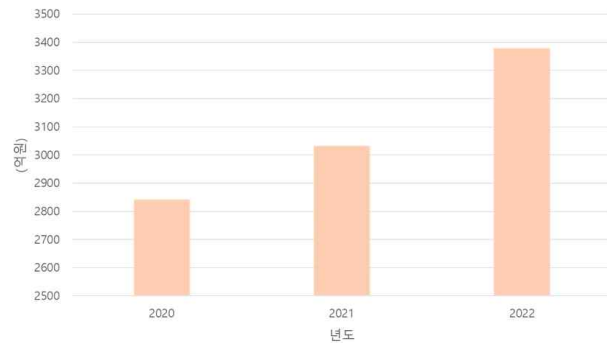
IT 기술이 발전하며 소프트웨어의 중요성 또한 크게 증가했다. 소프트웨어는 컴퓨터, 스마트폰, 자동차 심지어 현재는 냉장고, 세탁기 등 생활 가전속에 탑재되어 있을 만큼 소프트웨어는 우리 삶에 밀접해있다. 이러한 소프트웨어 개발은 최근 OSS(Open Source Software)를 이용해 개발되고 있다. OSS는 소스코드가 공개되어 누구나 자유롭게 무료로 사용 및 수정, 재배포 등이 가능한 소프트웨어이다[*1, 2]. 개발 시 특정 기능을 수행하는 OSS를 자신의 프로그램에 알맞게 수정하여 사용할 수 있다. 이로 인해 개발 시간과 비용을 획기적으로

국내 기업 OSS 활용률



<그림 1-1-1> 국내 기업 OSS 활용률

국내 OSS 시장 규모



〈그림 1-1-2〉 국내 OSS 시장 규모

로 감소시킬 수 있어 현대 개발 환경에 적극적으로 사용되고 있다.

정보통신산업진흥원에 따르면 국내 기업 중 OSS를 사용하는 기업 비율은 〈그림 1-1-1〉과 같다. 2020년 58.8%였던 OSS 활용률은 2022년 67.3%로 약 10% 상승한 모습을 보이고 있다. 또한, 국내 OSS 시장 규모는 〈그림 1-1-2〉에서 보이고 있듯이 2020년 약 2,843억원에서 2022년 약 3,380억원으로 성장한 모습을 보이고 있다.

하지만 OSS는 조건 없이 자유롭게 사용할 수 있는 것은 아니다. OSS를 사용하기 위해선 OSS의 사용 규정인 OSS 라이선스를 준수해야 한다. 만약 OSS 라이선스를 준수하지 않을 경우, 저작권 위반으로 인해 다양한 피해가 발생한다. 예를 들어 상업적 목적으로 개발한 소프트웨어의 판매를 중지하거나, 소프트웨어 코드 전체를 공개해야 하는 등의 피해가 발생할 수 있다. 또한, 저작권 위반으로 인해 법적 처벌을 받을 수 있다[*]. 그러므로 OSS를 사용 시 OSS 라이선스 준수는 반드시 이루어져야 한다.

OSS 라이선스의 종류로는 대표적으로 BSD(Berkeley Software Distribution), GPL(General Public License), MPL(Mozilla Public License), 그 외 다양한 라이선스가 존재한다[*]. 각 라이선스는 해당 라이선스만의 규정을 보유하고 있으며 상호 호환되는 라이선스가 존재하지만, 상호 호환되지 않는 라이선스 또한

존재한다. 만약 상호 호환되지 않는 라이선스를 혼용 시 저작권 위반으로 인해 프로그램 리콜, 법적 처벌 등의 피해가 발생할 수 있다.

그러나 다수의 개발자가 협업하여 소프트웨어를 개발하는 현대 개발 환경의 특징상, 다른 개발자가 사용한 OSS를 식별하지 못한 채로 상호 호환 불가능한 OSS를 사용하여 피해를 볼 수 있다. 이 경우 OSS 라이선스 준수 위반으로 인해 앞서 언급한 다양한 피해가 발생할 수 있다. 마이크로소프트의 경우, 코파일럿 학습 시 사용한 OSS의 라이선스 의무사항을 준수하지 않아 피소된 사례가 존재한다[*].

따라서, 이러한 피해를 방지하기 위해 개발 시 소프트웨어에 사용한 OSS를 식별하기 위한 방법이 필요하다. 이를 소스코드 유사도 검사를 활용하여 소스코드 내 OSS를 효과적으로 식별할 수 있다. 소스코드 유사도 검사는 두 개의 소스코드를 비교하여 두 소스코드가 얼마나 유사한지 계산하는 기법이다. 이를 활용하여 소프트웨어 내 동일한 소스코드를 식별하거나, 두 소스코드 내 특정 부분이 유사한지 확인할 수 있다. 또한, 소스코드 내 재사용된 동일한 함수, 변수 등을 확인하여 소스코드 유지보수 측면에서도 이를 활용할 수 있다. 현재 일반적으로 소스코드 유사도 검사는 텍스트 기반의 소스코드 유사도 검사로 진행되며 교육, 표절 검사, 악성코드 탐지 분야 혹은 코드 유지보수 등 다양한 환경에 사용되고 있다[*-*].

일반적인 텍스트 기반의 소스코드 유사도 검사는 소스코드 자체의 유사도, 즉 두 소스코드의 단어, 구조 등이 얼마나 유사한지 또는 표절 유무 등을 검사하기 위해 사용되는 기법이다[*]. 텍스트 기반의 소스코드 유사도 검사는 유사도 검사를 위해 소스코드에 추가적인 변환을 수행하지 않기에 구현이 간단하며 복잡한 처리가 필요하지 않다는 장점이 존재한다. 또한, 별도의 중간 추상화 단계가 필요하지 않아 대부분의 프로그래밍 언어에 적용할 수 있다[*]. 하지만 텍스트 기반의 소스코드 유사도 검사의 경우, 소스코드의 단어를 변경하거나 구조를 변경하는 등의 수정을 통해 유사도 검사를 회피할 수 있다는 단점이 존재한다[*].

이를 해결하기 위한 방법 중 하나로 소스코드를 그래프로 변환하여 소스코드 그레프의 유사도를 검사하는 기법이 존재한다[*]. 그래프 기반의 소스코드

유사도 검사는 텍스트 기반의 소스코드 유사도 검사에서 발생했던 소스코드 단어 변경 및 소스코드 구조 변경 등을 통한 유사도 검사 회피를 효과적으로 방지할 수 있다[*]. 그래프 기반의 소스코드 유사도 검사는 두 소스코드의 텍스트가 얼마나 유사한지 검사하는 것이 아닌, 두 소스코드의 문맥적 혹은 구조적 특징이 얼마나 유사한지 검사하는 기법이다.

텍스트와 달리 그래프로 변환된 소스코드는 소스코드의 단어, 구조를 변경해도 소스코드의 문맥적, 구조적 특징은 동일하다. 이를 통해 기존 텍스트 기반의 소스코드 유사도 검사에서 발생한 문제점을 효과적으로 해결할 수 있다. 또한, 소스코드를 시각화할 수 있어 사용자가 소스코드의 구조를 파악하는 데 도움을 줄 수 있다. 하지만 그래프 기반의 소스코드 유사도 검사는 소스코드를 그래프로 변환하는 작업이 필요하고, 텍스트보다 상대적으로 크기가 큰 그래프의 유사도를 비교한다. 그렇기에 텍스트 기반의 소스코드 유사도 검사에 비해 높은 계산 비용을 가진다는 단점이 존재한다[*]. <표 1-1-1>은 텍스트 기반의 소스코드 유사도 검사와 그래프 기반의 소스코드 유사도 검사의 비교 분석표이다.

이러한 그래프 기반의 소스코드 유사도 검사의 높은 계산 비용을 개선하기 위한 다양한 방법 중 휴리스틱 알고리즘이 존재한다. 휴리스틱 알고리즘은 기존 정확한 값을 계산하는 방식과는 달리, 근사값을 도출하여 기존 그래프 유사도 검사보다 빠른 시간 내에 유사도 검사를 수행할 수 있도록 한다. 즉, 그

<표 1-1-1> 소스코드 유사도 검사 기법 비교분석표

	텍스트 기반	그래프 기반
추가 변환 과정	불필요	필요
적용 범위	대부분의 프로그래밍 언어에 적용 가능	일부 기법은 특정 프로그래밍 언어만 적용 가능
유사도 검사 회피	함수명, 변수명, 구조 변경 등을 통해 회피 가능	함수명, 변수명, 구조 변경 등에 내성을 가짐
장점	- 간단한 구현 및 처리과정	- 소스코드의 문맥적/구조적 특징을 기반으로 정확한 유사도 검사 가능
단점	- 간단한 수정을 통해 유사도 검사 회피 가능	- 소스코드 그래프 변환 과정 필요 및 높은 계산 비용

래프 유사도 검사 시 모든 경우의 수를 확인 후 결과를 도출하는 기존 방식과 달리 두 그래프의 대략적인 유사도를 비교하여 빠른 시간 내에 유사도 근사값을 도출하는 방식이다. 이를 통해 빠른 시간 내에 유사도 검사를 수행할 수 있고, 계산 복잡성이 감소해 기존 그래프 유사도 검사 방식보다 상대적으로 더욱 큰 그래프의 유사도를 검사할 수 있다는 장점이 존재한다. 하지만 일부 k -walk 기반의 휴리스틱 알고리즘은 k -walk 생성 과정 중 Tottering 현상이 발생할 수 있다. Tottering 현상은 같은 노드를 반복하여 방문하는 현상으로써 이에 대한 자세한 설명은 2.5절에서 진행한다.

본 논문에선 이러한 소스코드를 그래프로 변환하여 소스코드 유사도 검사를 수행하는 그래프 기반의 소스코드 유사도 검사 기법을 제안한다. 또한 그래프 기반의 소스코드 유사도 검사의 높은 계산 복잡성을 해결하기 위한 휴리스틱 알고리즘을 접목 시 발생할 수 있는 Tottering 현상을 방지하는 새로운 k -walk 생성 알고리즘을 제안하여 더욱 효율적인 OSS 유사도 검사를 위한 소스코드 유사도 검사 기법을 제안하고자 한다. 본 연구의 목표는 이하 절에서 자세히 설명한다.

1.2 연구 목표

본 연구의 목표는 소프트웨어 내 사용된 OSS를 효과적으로 식별하기 위한 새로운 소스코드 유사도 검사 기법을 제안하는 것이다. 기존 텍스트 기반의 소스코드 유사도 검사의 단점을 극복하기 위해 그래프 기반의 소스코드 유사도 검사 기법을 적용한다. 그래프 기반의 유사도 검사 기법은 대표적으로 의미론 기반의 소스코드 유사도 검사, 추상 구문 트리 기반의 소스코드 유사도 검사, 소스코드 흐름도 기반의 소스코드 유사도 검사 기법이 존재한다. 하지만 기존 그래프 기반의 소스코드 유사도 검사 기법은 각 기법들의 고유한 단점 및 한계로 인해 소스코드 유사도 검사에 적합하지 않다.

본 연구 목표는 기존 그래프 기반의 유사도 검사 기법들의 단점 및 한계를 극복하기 위해 PDG(Program Dependence Graph)를 활용한 PDG 기반의 소스코드 유사도 검사 기법을 제안하는 것을 목표로 한다. 소스코드를 PDG로 변

제 2장 관련 연구

환 후, 변환된 PDG를 그래프 편집 거리 알고리즘을 이용해 두 그래프의 유사도를 비교한다. 그래프 기반의 소스코드 유사도 검사에 PDG를 활용하여 소스코드의 구조적 특징을 파악하고, 소스코드의 흐름을 효과적으로 파악할 수 있다. 또한, 특정 소스코드에 구애받지 않으므로 다양한 프로그래밍 언어를 지원할 수 있다.

하지만 PDG를 활용하여 그래프 유사도 계산 시 높은 계산 비용이 발생한다. 이는 기존 그래프 편집 거리 알고리즘의 높은 계산 복잡도로 인해 발생하는 문제이다[*]. 본 논문에선 이를 해결하기 위해 k -walk 기반의 휴리스틱 알고리즘을 적용해 높은 계산 비용 문제를 해결하고자 한다. k -walk 기반의 휴리스틱 알고리즘은 그래프 편집 거리 알고리즘의 계산 비용을 개선하기 위한 휴리스틱 알고리즘이다[*]. 해당 방식은 그래프 편집 거리 근사값을 빠르게 계산할 수 있다. 그러나 기존 k -walk 기반의 휴리스틱 알고리즘은 Tottering 현상이라는 문제점이 존재한다. 본 논문에선 이 Tottering 현상을 방지하는 새로운 k -walk 생성 알고리즘을 제안한다.

따라서 본 제안방식은 PDG를 활용해 기존 그래프 기반의 소스코드 유사도 검사 기법들과 비교해 소스코드의 구조적 특징 및 의미를 함께 파악할 수 있으며 다양한 프로그래밍 언어를 지원할 수 있는 유사도 검사 기법과 기존 k -walk 기반의 휴리스틱 알고리즘의 k -walk 생성 과정을 개선한 새로운 k -walk 생성 알고리즘을 제안함으로써 효율적인 OSS 유사도 검사를 수행할 수 있도록 한다.

본 장에서는 본 연구를 위해 필요한 기본 지식과 관련 연구에 대해 설명한다. 2.1절 그래프 기반의 소스코드 유사도 검사 알고리즘에서 그래프 기반의 소스코드 유사도 검사 알고리즘의 개념에 대해 설명한다. 또한, 이하 절에서 대표적인 그래프 기반의 소스코드 유사도 검사 알고리즘의 개념 및 장단점 등을 기술한다. 이후, 본 제안방식을 이해하기 위한 기본 지식인 PDG, 그래프 편집 거리 알고리즘, 휴리스틱 알고리즘, Tottering 현상에 대하여 설명한다.

2.1 그래프 기반의 소스코드 유사도 검사 알고리즘

그래프 기반의 소스코드 유사도 검사는 소스코드를 특정 그래프로 변환한 후, 변환된 소스코드 그래프를 입력으로 하여 두 그래프의 유사도를 측정하는 기법이다[*]. 소스코드를 그래프로 표현할 경우, 소스코드의 구조와 종속성 등을 나타낼 수 있다. 그래프의 노드는 일반적으로 클래스, 함수, 변수 등과 같은 코드를 구성하는 요소이며, 엣지는 코드 구성 요소 간의 관계를 나타낸다.

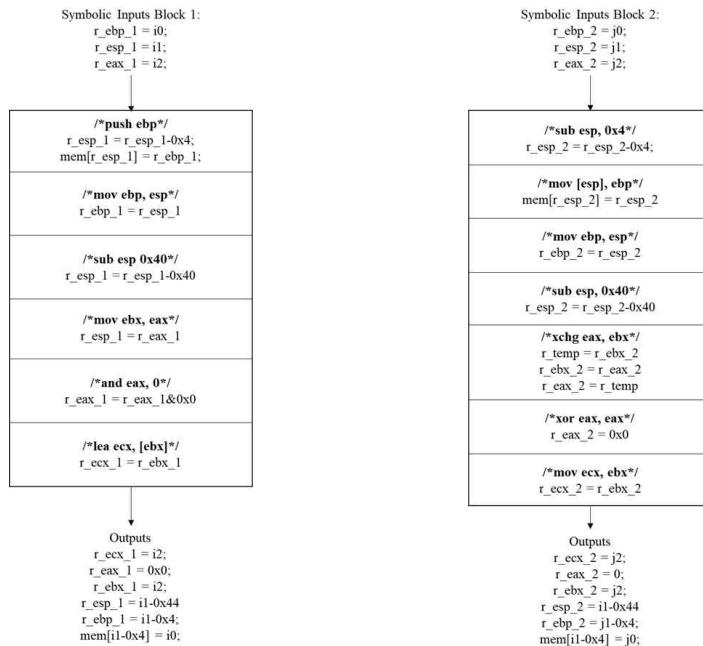
<표 II-1-1> 그래프 기반의 소스코드 유사도 검사 알고리즘 비교분석표

	장점	단점
의미론 기반	- 실제 기능과 논리 기반으로 유사도를 평가해 높은 정확도를 보임 - 코드가 난독화되어도 같은 기능일 경우 탐지 가능	- 높은 계산 비용 - 특정 프로그래밍 언어만 지원
추상 구문 트리 기반	- 코드의 구조적 유사성을 정확히 평가 가능 - 소스 코드의 구문적 구조를 분석해 효과적인 비교 가능	- 코드의 의미나 동작 과정을 고려하지 못함 - 높은 계산 비용
소스코드 흐름도 기반	- 소스코드 동작 흐름을 기반으로 하여 로직 분석에 용이 - 코드의 구조적, 논리적 유사도 고려 가능	- 소스코드를 흐름도로 변환 및 비교 하는 과정에서 높은 계산 비용 발생 - 변수나 데이터 고려 x

그래프 기반의 소스코드 유사도 검사 기법은 대표적으로 의미론 기반의 소스코드 유사도 검사, 추상 구문 트리 기반의 소스코드 유사도 검사, 소스코드 흐름도 기반의 소스코드 유사도 검사 기법이 존재한다. 각 기법에 대한 자세한 설명은 이하 절에서 설명한다. 또한, 각 기법에 대한 기존 연구들은 4장 기존방식 분석에서 자세히 설명한다. <표 II-1-1>는 각 기법의 장단점을 분석한 비교분석표이다.

2.1.1 의미론 기반의 소스코드 유사도 검사

의미론 기반의 소스코드 유사도 검사는 두 소스코드의 의미, 즉 소스코드가 실제로 동작하는 과정에 대한 유사도를 검사하는 방식이다[*]. 해당 방식은 소스코드의 작성 형태를 분석하는 것보다 소스코드가 실제 실행 시 어떻게 실행되는지 등 실제 기능과 논리를 분석해 코드의 유사도를 검사한다.



<그림 II-1-1-1> 기호적 표현으로 변환된 소스코드 예시

의미론 기반의 소스코드 유사도 검사는 소스코드의 실제 동작 과정을 기반으로 유사도를 측정하기에 다른 기법보다 정확도가 높다. 또한, 소스코드 난독화를 적용하여 소스코드 유사도 탐지를 회피하려 해도 난독화된 소스코드의 기능이 동일하면 이를 식별할 수 있다. 의미론 기반의 소스코드 유사도 검사는 주로 소스코드를 기호적 표현으로 변환해, 이를 기반으로 소스코드 경로를 생성해 유사도를 비교한다. <그림 II-1-1-1>은 기호적 표현으로 변환된 소스코드의 예시이다.

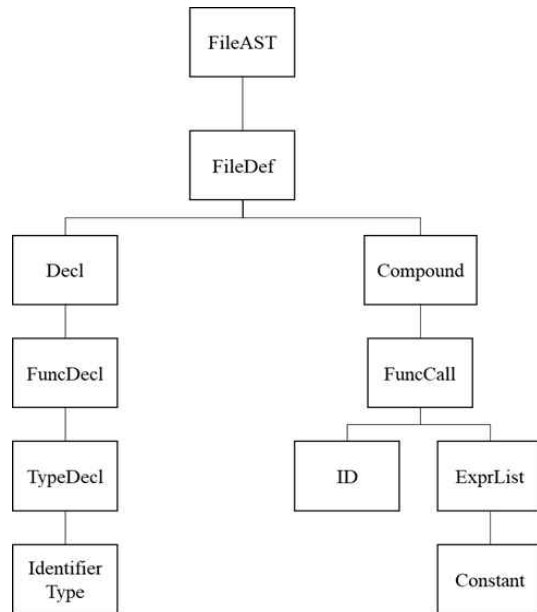
하지만 의미론 기반의 소스코드 유사도 검사는 소스코드의 실행 과정, 논리 등을 분석함에 따라 다른 방식보다 높은 계산 비용이 발생한다[*]. 또한, 현재 의미론 기반의 소스코드 유사도 검사 기법이 최적화된 프로그래밍 언어가 제한되어 있다. 의미론 기반의 소스코드 유사도 검사 기법은 현재 주로 C, C++, Java, Python 등에 최적화되어 있어 다양한 프로그래밍 언어를 지원할 수 없다는 한계가 존재한다.

2.1.2 추상 구문 트리 기반의 소스코드 유사도 검사

추상 구문 트리란 소스코드의 논리적 구조를 트리 형식으로 표현한 구조를 의미한다[*]. 추상 구문 트리 기반의 소스코드 유사도 검사는 두 소스코드의 추상 구문 트리를 생성하여 각 추상 구문 트리의 노드 해시 값을 비교하여 유사도를 측정한다. <그림 II-1-2-1>은 소스코드를 추상 구문 트리로 변환한 예시이다.

추상 구문 트리 기반의 소스코드 유사도 검사는 코드의 구문적 구조를 효과적으로 나타낼 수 있으므로 코드의 구조적 유사성을 정확하게 평가할 수 있다는 장점이 존재한다. 이를 통해 두 소스코드가 얼마나 비슷한 형식으로 이루어져 있는지 알 수 있다.

하지만 추상 구문 트리 기반의 소스코드 유사도 검사는 코드의 구조적 유사성을 주로 판단하기에 코드가 가진 의미를 파악하기 힘들다는 단점이 존재한다. 즉, 코드가 실제로 수행하는 작업에 대한 유사도는 측정할 수 없다. 이로 인해 같은 동작을 수행하는 다른 형태의 코드에 대한 유사도는 판단할 수 없다는 한계점이 존재한다. 또한, 높은 계산 비용을 갖는 문제점이 존재한다[*].

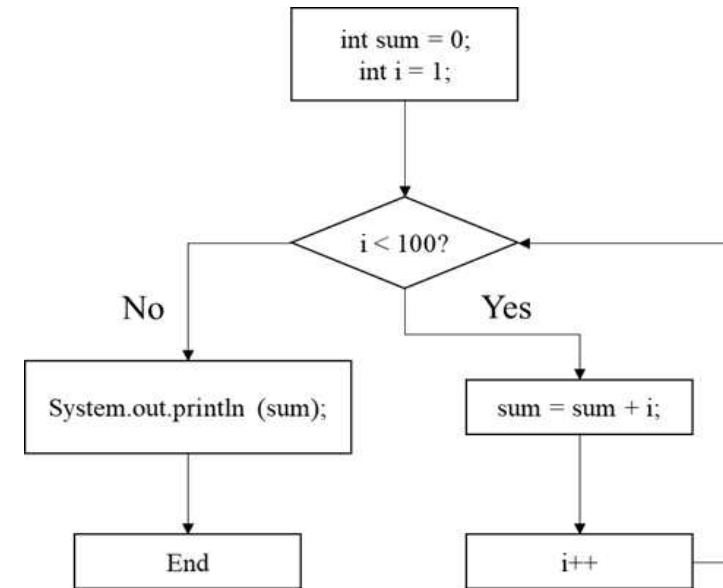


〈그림 II-1-2-1〉 추상 구문 트리 예시

2.1.3 소스코드 흐름도 기반의 소스코드 유사도 검사

소스코드 흐름도란 소스코드의 동작 과정을 간단한 기호와 도형을 이용해 순서대로 표현한 그래프를 의미한다. 즉, 소스코드가 동작하는 과정을 사람이 이해하기 쉽게하기 위해 그래프 형식으로 표현한 소스코드 그래프이다. 소스코드 흐름도 기반의 소스코드 유사도 검사는 이러한 소스코드 흐름도를 이용하여 두 소스코드의 유사도를 측정한다[*]. 일반적으로 소스코드를 PDG로 변환한 후, 이를 일반화하여 소스코드 흐름도를 생성한다. 이후 생성된 소스코드 흐름도를 활용하여 두 소스코드 흐름도의 유사도를 측정한다. 〈그림 II-1-3-1〉은 소스코드를 소스코드 흐름도로 변환한 예시이다.

소스코드 흐름도 기반의 소스코드 유사도 검사는 소스코드 흐름도를 통해 소스코드의 실행 흐름을 시각적으로 나타내어 소스코드의 로직을 시각적으로 비교할 수 있다는 장점이 존재한다. 또한, 코드의 텍스트적 차이가 아닌 소스



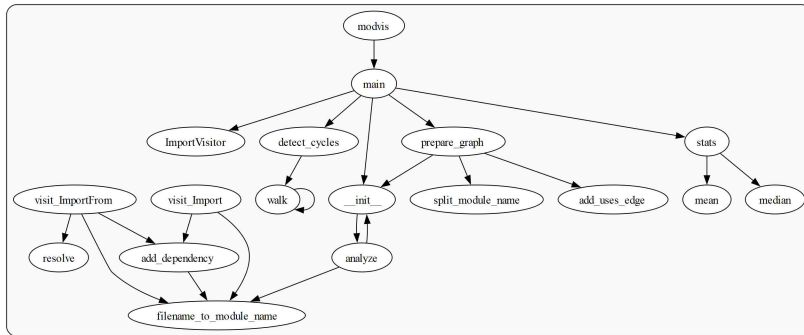
〈그림 II-1-3-1〉 소스코드 흐름도 예시

코드의 구조와 로직의 유사도를 기반으로 유사도를 측정하기 때문에 코드의 의미 또한 고려할 수 있다는 장점이 존재한다.

하지만 소스코드를 소스코드 흐름도로 변환하고 비교하는 과정에서 높은 계산 오버헤드가 발생한다[*]. 추가적으로, 변수나 데이터를 고려하지 않기에 소스코드 유사도 검사에 영향을 미칠 수 있다는 한계점이 존재한다.

2.2 PDG

PDG란 Program Dependence Graph의 약자이며 프로그램 종속성 그래프를 의미한다. PDG는 코드 내에 데이터 종속성과 제어 종속성을 나타내는 그래프로써 프로그램의 관련 데이터 흐름 관계를 명시적으로 나타내는 그래프이다[*]. 데이터 종속 그래프는 소스코드에 존재하는 정의-사용 관계를 나타내는 그래프이며, 제어 흐름 그래프는 소스코드의 제어 흐름 관계를 명시적으로 표현한 그래프이다. 〈그림 II-2-1〉은 소스코드를 PDG로 변환한 예시이다.



〈그림 II-2-1〉 Program Dependence Graph 예시

여기서 종속성이란 한 요소가 다른 요소에 의존하는 관계이다. 소스코드에서 종속성은 두 가지 별개의 효과로 인해 발생한다. 첫 번째 효과는 두 명령문이 반전되면 한 명령문에 나타내는 변수가 잘못된 가질 수 있을 때마다 두 명령문 사이에 종속성이 발생하는 것이다. 두 번째 효과는 명령문과 그 값이 명령문의 실행을 즉시 제어하는 술어 사이에서 종속성이 발생한다[*].

PDG는 이러한 종속성을 이용해 소스코드 내 계산적으로 중요한 부분을 연결하여 다른 방식보다 적은 시간으로 변환 작업이 수행 가능하다. 그래프 내의 노드는 문장과 조건 표현식을 의미하고, 노드에 인접한 엣지는 노드의 연산이 의존하는 데이터 값과 연산의 실행이 의존하는 제어 조건을 나타낸다.

이러한 PDG를 그래프 기반의 소스코드 유사도 검사 기법에 적용할 경우 기존 방식들 대비 유사한 소스코드를 더욱 정확히 식별할 수 있다[*]. 이는 PDG를 이용해 소스코드 유사도 검사를 진행할 시 소스코드의 구조 및 동작 과정 등을 고려할 수 있기 때문이다.

본 논문에선 이러한 PDG를 기반으로한 소스코드 유사도 검사 기법을 제안한다. 소스코드를 PDG로 변환한 후, 변환된 PDG를 개선된 k -walk 기반의 휴리스틱 알고리즘을 이용하여 두 소스코드의 유사도를 검사한다.

2.3 그래프 편집 거리

그래프 편집 거리(Graph Edit Distance) 알고리즘은 그래프 A를 그래프 B로 변환하기 위한 편집 경로 중 최소 편집 비용을 계산하는 알고리즘이다. 그래프 편집 거리는 완전 그래프, 연결 그래프, 방향 그래프 등 다양한 그래프에 적용할 수 있다. 그래프 편집 거리 알고리즘은 이러한 그래프의 노드, 엣지 또는 노드와 엣지 모두를 고려하여 두 그래프 간의 유사도를 측정한다.

여기서 그래프란 노드와 엣지로 구성된 자료구조를 의미하며 $G=(V,E)$ 의 형태로 정의된다. V 는 노드 집합을 의미하며 $V=\{v_1, v_2, \dots, v_m\}$ 로 정의된다. 여기서 v_n 은 그래프를 구성하는 각각의 노드를 의미한다. E 는 엣지 집합을 의미하며 $E=\{(v_1, v_2), (v_1, v_3), \dots, (v_{m-1}, v_m)\}$ 와 같이 $E \subseteq V \times V$ 로 정의된다[*].

그래프 편집 거리는 이러한 그래프의 구조적 특징을 이용하여 그래프 G 를 그래프 H 로 변환하기 위한 편집 거리를 계산하는 알고리즘이다. 여기서 편집 거리란 노드의 수정, 삽입, 삭제와 엣지의 수정, 삽입, 삭제 등과 같이 그래프 구성 요소를 편집하는 비용을 의미한다. 각 편집 연산의 정의는 아래와 같다.

- 노드 수정: G 의 노드를 변환하여 H 의 노드와 일치시키는 작업
- 노드 삽입: G 에 존재하지 않는 노드를 H 의 구조와 일치하도록 새로운 노드를 삽입하는 작업
- 노드 삭제: G 에 존재하는 노드를 제거하여 H 의 구조에 맞게 수정하는 작업
- 엣지 수정: 두 노드 사이에 엣지를 다른 엣지로 교체하는 작업
- 엣지 삽입: 두 노드 사이에 새로운 엣지를 추가하는 작업
- 엣지 삭제: 기존에 존재하던 엣지를 삭제하는 작업

그래프 편집 거리는 이러한 편집 연산을 이용해 다양한 편집 거리를 산출해 이 중 편집 거리가 가장 짧은, 즉 편집 비용이 가장 작은 경우의 수를 계산하는 알고리즘이다.

그래프 편집 거리의 동작 과정은 다음과 같다.

Step1. 그래프 편집 거리 알고리즘을 위한 편집 연산을 정의한다. 편집 연

산은 노드 수정, 삽입, 삭제와 엣지 수정, 삽입, 삭제로 구분된다.

Step2. 편집 연산 정의 후 각 편집 연산에 소모되는 비용을 할당한다. 노드 수정, 삽입, 삭제와 엣지 수정, 삽입, 삭제 연산에 필요한 비용을 정의해 편집 거리 계산에 활용한다. 그래프 편집 거리 알고리즘은 이 비용을 기반으로 최소 편집 비용을 계산하는 것이 목적이다.

Step3. 다양한 편집 경로를 계산하여 편집 비용이 가장 적은 경로를 찾는다. 이때 전사적 탐색, A* 알고리즘 등 다양한 탐색 알고리즘을 활용하여 최소 편집 경로를 계산한다.

이러한 과정을 통해 두 그래프 간의 최소 편집 비용을 계산하여 두 그래프가 얼마나 유사한 그래프인지 수치적으로 확인할 수 있다. 그래프 편집 거리 알고리즘은 모든 편집 경로를 확인하여 그 중 최소 편집 비용이 발생하는 결과를 기반으로 편집 비용이 결정된다.

이처럼 두 그래프를 비교하기 위한 방식은 그래프 편집 거리 외에도 다양한 알고리즘이 존재한다. 그래프 편집 거리 외에 다른 방식을 이용해 그래프 유사도를 측정할 수 있다. AI를 이용해 그래프 유사도를 측정하는 AI 기반의 그래프 유사도 검사 기법, 커널 함수를 사용해 그래프 특징을 벡터로 변환하여 그래프 유사도를 검사하는 그래프 커널 기법, 서브 그래프 단위로 그래프를 비교해 그래프 유사도를 검사하는 서브 그래프 동형성 기법, 그래프를 다차원 벡터 공간으로 표현한 뒤, 벡터로 표현된 그래프를 비교하는 그래프 임베딩 기법이 존재한다[*-*]. 그러나 해당 방식들은 각각의 단점에 더불어 그래프 구조적 유사도를 파악하는데 특화되지 않았다는 한계점이 존재한다. 이에 대한 비교분석표는 <표 II-3-1>과 같다.

하지만 그래프 편집 거리는 원본 그래프를 그대로 사용하기에 정보 손실이 없으며 다양한 그래프를 접목할 수 있어 높은 유연성을 가진다. 또한, 그래프 편집 거리는 정보 손실이 적어 그래프의 구조를 직관적으로 비교할 수 있다는 장점이 존재한다. 이러한 그래프 편집 거리의 장점들로 인해 그래프를 비교 및 분석하는 다양한 분야에서 이를 활발히 활용하고 있으며 그래프 편집 거리는 그래프 비교 및 분석 분야에서 매우 유용한 값을 제공한다[*]. 현재 그래프

편집 거리는 그래프 분류, 컴퓨터 비전, 패턴 인식 등 다양한 분야에서 사용된다. 컴퓨터 비전 분야에선 이미지 내에 객체 인식에 사용되며 이미지의 구조적 특징은 분석해 대표 그래프를 생성하고 이를 비교한다[*].

하지만 그래프 편집 거리는 모든 편집 경로를 확인해야하기 때문에 시간 복잡도가 크다는 한계점이 존재한다. 그래프 편집 거리는 모든 편집 경로를 확인해야하므로 NP(Non-deterministic Polynomial time)-Hard 문제에 속한다. NP-Hard 문제란 최적의 해결책을 제공할 수 없는 비결정적 다항식 문제를 의미한다. 그렇기 때문에 다항 시간 내에 문제를 해결할 수 없어 시간 복잡도가 크다. 특히 노드와 엣지가 많거나 복잡한 그래프에선 이러한 문제가 더욱 크게 발생한다.

<표 II-3-1> 그래프 유사도 검사 기법 비교분석표

	장점	단점
그래프 편집 거리	- 세부적인 그래프 구조적 특성 반영 가능 - 각종 편집 연산을 기반으로 직관적인 해석 가능	- 높은 계산 비용 - 대규모 그래프에 부적합
AI 기반 그래프 유사도	- 그래프 패턴 학습 가능 - 대규모 그래프에 적합	- 결과 해석의 어려움 - 학습 모델 생성 필수 - 모델 선택 및 학습의 어려움
그래프 커널	- 빠른 시간 내 그래프 구조 분석 가능 - 다양한 커널 함수를 활용해 높은 유연성	- 커널 함수에 따른 성능 편차 - 그래프의 세부적 특성 반영 불가
서브 그래프 동형성	- 서브 그래프 단위 비교 시 높은 성능 - 서브 그래프 구조적 일치성을 정확히 탐지	- 전체적인 그래프 구조 비교 시 부적합 - 높은 계산 비용 - 대규모 그래프에 부적합
그래프 임베딩	- 고차원 그래프 구조를 저차원 벡터로 변환해 효율적인 비교 - 벡터 기반 연산을 통한 빠른 계산 속도 - 대규모 그래프에 적합	- 벡터 임베딩 과정 중 그래프 정보 손실 발생 - 그래프의 세부적 특성 반영 불가

이러한 그래프 편집 거리의 단점을 개선하기 위해 다양한 연구가 진행되었다. 그 중, 휴리스틱 알고리즘은 정확한 값을 계산하는 것이 아닌 근사값을 계산하여 빠른 시간 내에 근사값을 도출할 수 있도록 한다. 휴리스틱 알고리즘에 대해선 2.4절에서 자세히 설명한다.

2.4 휴리스틱 알고리즘

휴리스틱 알고리즘은 특정 알고리즘에서 최적의 해를 찾는 대신, 허용할 수 있는 수준의 오차를 가진 근사값을 도출하는 알고리즘이다. 휴리스틱 알고리즘은 특정 문제의 시간 복잡도가 매우 높거나 다항 시간 내에 최적해를 찾는 것이 불가능할 때 주로 사용된다.

휴리스틱 알고리즘은 최적화 문제 혹은 탐색 문제 등에서 문제의 구조적 특성을 활용해 탐색 공간을 줄이고, 줄어든 탐색 공간 내에서 가장 적절한 근사값을 찾는다. 즉, 휴리스틱 알고리즘은 정확한 값을 찾아내는 것이 알고리즘 목표가 아닌, 허용할 수 있는 오차를 포함한 값을 빠른 시간 내에 찾아내는 것을 목표로 한다.

그래프 편집 거리를 위한 휴리스틱 알고리즘은 모든 편집 경로를 확인하는 기존 그래프 편집 거리와는 달리, 그래프의 탐색 공간을 줄여 줄어든 그래프 내에서 편집 경로를 확인하는 알고리즘이다. 그래프 편집 거리를 위한 휴리스틱 알고리즘은 크게 3가지로 분류된다. LSAP(Linear Sum Assignment Problem with Error correction)-GED(Graph Edit Distance), LP(Linear Programming)-GED, LS(Local Search)-GED가 존재한다.

LSAPE-GED는 오류 수정이 포함된 선형 합 할당 문제로 변환해 그래프 편집 거리의 근사값을 계산하는 휴리스틱 알고리즘 중 하나이다. LSAPE-GED는 그래프 편집 거리의 상한을 계산할 수 있으며 일부 기법은 하한 또한 계산할 수 있다. LP-GED는 그래프 편집 거리에 대한 혼합 정수 프로그래밍 공식의 선형 프로그래밍을 사용해 그래프 편집 거리의 상한과 하한을 계산하는 기법이다. LS-GED는 그래프 편집 거리의 상한을 계산하기 위한 기법이다. 노드와 엣지의 편집 비용을 고려해 초기 노드 매핑을 생성하거나 무작위로 구성 후,

지역 탐색을 통해 해당 매핑을 개선한다[*]. 이 중 본 제안방식에서 사용되는 LSAP-GED에 대한 설명은 이하 절에서 설명한다.

2.4.1 LSAP-GED

LSAPE-GED는 기존 선형 합 할당 문제에 오류 수정 개념을 포함하여 그래프 편집 거리의 근사값을 계산하는 기법이다. LSAP에 대한 정의는 아래와 같다.

정의 1 (최초의 LSAP 정의[*]) LSAP(Linear Sum Assignment Problem)은 최대 매칭 X 에 대한 $K_{n,m} = (V_G, V_H, V_G \times V_H)$ 와 비용 행렬 C 가 주어진다. X 의 셀 $x_{i,j}$ 는 V_G 의 노드 v_{G_i} 가 V_H 의 노드 v_{H_j} 로 대체되는 비용을 의미하며, 대체 비용은 $c_{i,j}$ 로 정의된다. 즉, LSAP는 V_G 에 포함된 노드들을 최소한의 비용으로 V_H 의 노드로 변경할 때 발생하는 비용을 계산한다.

이 때, $n < m$ 일 경우, V_H 에는 V_G 에 대응되지 않는 노드, 즉 V_H 에만 존재하는 노드가 반드시 한 개 이상 존재한다. 이 경우 V_G 에 대응되지 않는 V_H 의 노드가 존재하므로 기존 LSAP를 이용해 이 노드들을 처리할 수 없다. 이러한 문제를 해결하기 위해 오류 수정이 포함된 선형 합 할당 문제, 즉 LSAP를 사용할 수 있다.

LSAPE는 V_G 와 V_H 의 노드에 각각 더미 노드 ϵ 를 포함하여 $V_{G_\epsilon} = V_G \cup \{\epsilon\}$ 와 $V_{H_\epsilon} = V_H \cup \{\epsilon\}$ 로 정의된다. 엣지 집합 $M = V_{G_\epsilon} \cup V_{H_\epsilon} \cup V_{G_\epsilon} \times V_{H_\epsilon}$ 은 오류 수정 매칭이라 하며, $K_{n,m,\epsilon} = (V_{G_\epsilon}, V_{H_\epsilon}, V_{G_\epsilon} \times V_{H_\epsilon})$ 에만 속하지 않는 엣지 (ϵ, ϵ) 가 없다. 이는 더미 노드 간의 매칭이 존재하지 않는다는 것을 의미한다. 또한, 모든 V_{G_ϵ} 와 V_{H_ϵ} 의 노드가 M 에서 정확히 하나의 엣지와 인접하는 경우에만 존재한다. 이는 V_{G_ϵ} 에 속하는 각 노드가 반드시 V_{H_ϵ} 의 노드 중 한 노드와만 매칭을 가질 수 있다는 것을 의미한다.

Sub_C 가 모든 노드 대체를 의미하고, Rem_C 가 모든 노드 삭제를 의미하며, Ins_C 를 모든 노드 삽입이라고 정의할 때 오류 수정 매칭 집합 $\Pi_{n,m,\epsilon}$ 은 아래와 같다:

$$\Pi_{n,m,\epsilon} = \left\{ X = \{X_{sub}, X_{rem}, X_{ins}\} \in \{0,1\}^{(n+1) \times (m+1)} : \right. \\ \left. \forall j \in J, x_{\in j} + \sum_{i=1}^n x_{i,j} = 1, \forall i \in I, x_{\in i} + \sum_{j=1}^m x_{i,j} = 1 \right\}$$

이를 기반으로 생성된 LSAPE의 비용 행렬 C 는 아래와 같다:

$$C = \begin{bmatrix} C_{sub} & C_{rem} \\ C_{ins} & 0 \end{bmatrix}$$

여기서 C_{sub} 는 대체 비용, C_{rem} 은 삭제 비용, C_{ins} 는 삽입 비용을 의미한다. 이와 같은 비용 행렬 C 를 생성한 뒤, $K_{n,m} = (V_G, V_H, V_G \times V_H)$ 에 대한 삽입, 삭제, 대체 비용의 합이 최소인 경우의 수를 찾는 오류 수정 매칭 X^* 의 계산을 수행한다.

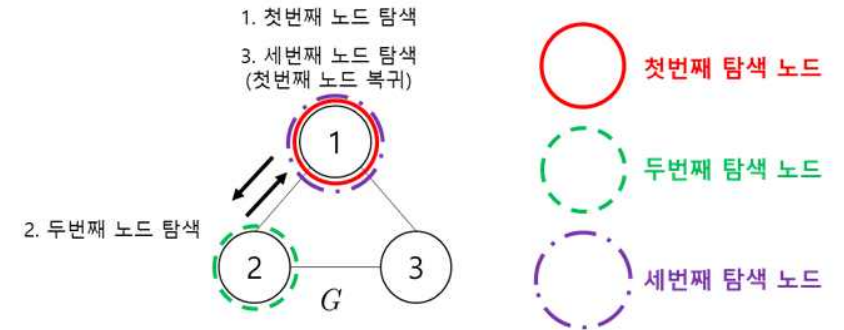
비용 행렬 $C \in \mathbb{R}^{(n+1) \times (m+1)}$ 이 주어지면 LSAPE는 다음과 같이 오류 수정 매칭 X^* 를 계산한다. $X^* \in \Pi_{n,m,\epsilon}$ 일 때, $X^* = \operatorname{argmin}_{X \in \Pi_{n,m,\epsilon}} L(X, C)$ 를 구한다. 이 때, $\operatorname{argmin}_{X \in \Pi_{n,m,\epsilon}} L(X, C)$ 은 최적의 매칭을 의미하고

$L(X, C) = \sum_{i=1}^n \sum_{j=1}^m c_{i,\epsilon} x_{i,\epsilon} + \sum_{j=1}^m c_{\epsilon,j} x_{\epsilon,j}$ 이며 총 비용을 의미한다. 즉, 주어진 비용 행렬에 대해 최소 비용을 가진 두 집합 간의 최적의 편집 비용을 계산하는 과정을 의미한다.

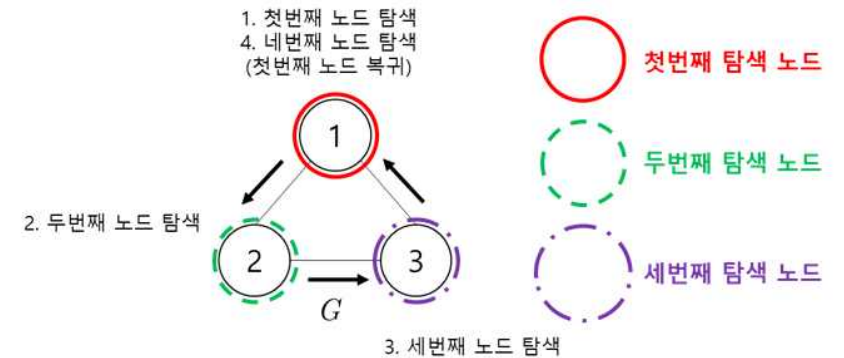
2.5 Tottering 현상

Tottering 현상이란 그래프 탐색 시 두 노드만을 번갈아가며 탐색하는 현상을 의미한다. 즉, 그래프 탐색 과정에서 특정 두 노드를 반복적으로 중복 방문하는 경우 이를 Tottering 현상이라 한다. 시작 노드를 기준으로 탐색한 후 인접한 다음 노드를 탐색한 뒤, 다음 탐색을 다시 시작 노드로 선정하는 경우 Tottering 현상이다. 시작 노드뿐만 아닌 특정 노드를 탐색한 후, 인접 노드를 탐색한 뒤 다시 해당 노드로 돌아오는 탐색 또한 중복 방문이므로 이는 Tottering 현상이 발생한 것이다. 그래프 탐색 시 <그림 II-5-1>은 Tottering 현상이 발생했을 때의 예시를 설명한 그림이다.

하지만 두 노드만을 번갈아가며 탐색하는 것과 달리 다른 노드를 거쳐 다시 특정 노드로 돌아오는 것은 Tottering 현상이 아닌 그래프의 구조 중 하나인



<그림 II-5-1> Tottering 발생 예시
사이클 구조라고 한다. 사이클 구조는 그림 *와 같으며 해당 그림처럼 노드 1,



<그림 II-5-2> 사이클 구조 예시

노드 2, 노드 3, 노드 4를 거쳐 다시 노드 1로 돌아오는 경우는 Tottering 현상이 아니다. <그림 II-5-2>는 그래프 탐색 시 사이클 구조를 설명한 그림이다.

그래프 탐색 과정에서 Tottering 현상이 발생할 경우 다양한 피해가 발생한다. 그래프 탐색에 불필요한 과정이 발생하여 실행 시간이 증가할 수 있다. 또한, 불필요한 탐색이 발생하여 유사도 검사 결과에 악영향을 미칠 수 있다.

제 3장 기존방식 분석

본 장에서는 그래프 기반의 소스코드 유사도 검사의 기존 연구에 대해 설명한다. 의미론 기반의 소스코드 유사도 검사, 추상 구문 트리 기반의 소스코드 유사도 검사, 소스코드 흐름도 기반의 소스코드 유사도 검사 기법의 대표적인 기법들에 대해 설명하고 해당 방식들의 한계점을 분석한다. 또한, 기존 k -walk 기반의 휴리스틱 알고리즘을 설명하고, 해당 방식에서 발생하는 한계점을 분석한다.

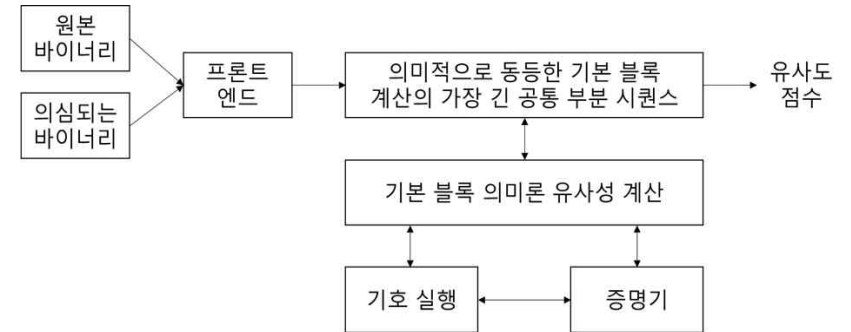
3.1 그래프 기반의 소스코드 유사도 검사 알고리즘

3.1.1 의미론 기반의 소스코드 유사도 검사

Luo et al.은 CoP라는 바이너리 지향 난독화 저항성을 가진 의미론 기반의 소스코드 유사도 검사 기법을 제안했다[*]. 해당 방식은 일반적인 의미론 방식에 최장 공통 부분 수열 기반의 퍼지 매칭을 결합한 방식이다. Luo et al.가 제안한 알고리즘의 구조는 <그림 III-1-1>과 같다. 해당 방식의 객체에 대한 설명 및 동작 과정은 다음과 같다.

I. 시스템 객체

- 원본 바이너리: 의심되는 소스코드가 표절한 소스코드임을 주장하는 프로그램의 바이너리 파일
- 의심되는 바이너리: 표절이 의심되는 프로그램의 바이너리 파일
- 프론트엔드: 두 개의 바이너리 파일을 전처리하는 객체. 원본 바이너리와 의심되는 바이너리를 분석해 이진 파일을 디스어셈블링한다. 이를 통해 바이너리 내부의 기본 블록을 추출함
- 기본 블록: 프로그램 제어 흐름 내 조건문 없이 순차적으로 실행되는 명령어 집합
- 의미적으로 동등한 기본 블록 계산의 가장 긴 공통 부분 시퀀스: 의미적으로 동일한 기본 블록들의 가장 긴 공통 부분 수열을 계산하는 객



<그림 III-1-1> Luo et al. CoP 구조

체. 원본 및 의심되는 바이너리의 기본 블록들을 분석하여 의미적으로 동일한 블록을 찾아 두 바이너리의 유사도를 계산함

- 기본 블록 의미론 유사성 계산: 두 바이너리의 기본 블록 간의 의미적 유사도를 측정하는 객체. 기호 실행과 증명기를 통해 기본 블록 간의 의미적 유사도를 측정함
- 기호 실행: 프로그램의 실행을 추상화해 입/출력 관계를 분석하는 객체. 각 프로그램의 기본 블록의 입/출력 관계를 기호로 표현하여 분석하여 블록의 실행 의미를 추출함
- 증명기: 기호 실행 결과를 기반으로 기본 블록 간의 의미적 유사도를 증명하는 객체. 원본 바이너리와 의심되는 바이너리의 기본 블록이 같은 기능을 수행하는지 증명함

II. 이진 코드 전처리

이진 코드 전처리 단계는 소스코드를 바이너리 코드로 변환한 후, 각 코드의 프로그램 실행 경로를 나타내는 CFG(Control Flow Graph)를 생성해 기본 블록을 추출하는 단계이다. 이후, 각 CFG의 기본 블록(분기 없는 코드 흐름)을 분석해 코드의 동작 과정을 분석한다. 이진 코드 전처리 단계는 아래와 같이 동작한다.

Step1. 두 개의 바이너리 파일(원본 바이너리, 의심되는 바이너리)을 입력한다.

Step2. 프론트엔드에서 바이너리 파일을 디스어셈블리하여 기본 블록을 생성하고 이를 통해 제어 흐름 그래프를 생성한다.

III. 기본 블록 의미적 유사도 계산

이진 코드 전처리 과정을 통해 두 바이너리 파일의 기본 블록을 추출하면 이후 기본 블록 간의 의미적 유사도를 계산한다. 해당 방식은 노이즈를 수용한 의미적 유사도를 계산한다. 여기서 의미적 유사도란 두 코드 블록이 구분적으로 동일하지 않아도 입/출력 관계가 의미적으로 동일하거나 유사한 척도를 의미한다. 즉, 변수의 이름이 다르거나 코드의 순서나 구조가 다르더라도 입력에 따른 출력이 동일하거나 유사한 경우를 의미적 유사도라 한다. 해당 방식의 기본 블록 의미적 유사도 계산 과정은 다음과 같다.

Step1. 두 $\alpha \leq \beta$ 인 변수 목록 $A = [a_0, a_1, \dots, a_\alpha]$ 와 $B = [b_0, b_1, \dots, b_\beta]$ 가 주어지고 B 의 변수 순열 $\pi(B)$ 를 정의한다.

Step2. 식 (*)를 통해 A 와 B 에 대한 쌍별 동등성을 계산한다. 여기서 A_i, B_i 는 각각 A 와 B 의 i 번째 변수를 의미하고 $\pi(B)$ 는 B 의 수열이다.

$$p(X, \pi(Y)) = \bigwedge_{i=0}^{\alpha} (A_i = \pi_i(B)) \quad (*)$$

Step3. 이후 각각의 입력 변수 목록과 출력 변수 목록을 이용해 출력 유사도를 비교한다. A_1, A_2 가 입력 변수 목록이고 B_1, B_2 가 출력 변수 목록이라고 가정하고 $|A_1| \leq |A_2|$ 라고 가정한다. 또한, 첫 블록은 원본 블록, 두 번째 블록은 의심되는 블록이라고 가정하면 식 (*)를 통해 출력 유사도를 비교한다

$$\forall b_1 \in B_1, \exists b_2 \in B_2, p(A_1, \pi(A_2)) \quad (*)$$

Step4. 위와 같은 과정을 통해 원본 블록의 각 출력 변수와 의심되는 블록의 입력 변수 조합 중 출력 유사도를 가진 출력 변수가 존재하는지 확인한다.

IV. 의미적으로 동일한 기본 블록들의 가장 긴 공통 부분 수열 계산

기본 블록 의미적 유사도 검사 후, 유사도가 특정 임계값을 넘긴 결과에 대해 의미적으로 동일한 기본 블록의 가장 공통 부분 수열을 계산하여 경로 임베딩 점수를 계산한다. 여기서 가장 공통 부분 수열이란 두 개의 블록에서 순서를 유지한 채 공통적으로 나타나는 가장 긴 부분 수열을 의미한다.

가장 긴 공통 부분 수열 계산 시 사용되는 파라미터는 다음과 같다.

- δ : LCS(Longest Common Subsequence) 점수를 저장하는 테이블
- r : 현재 테이블의 행을 나타내는 인덱스
- γ : LCS 검색 방향을 저장하는 배열
- σ : 중간 LCS 점수를 저장하는 배열
- n : 원본 프로그램의 선형 독립 경로 ρ 의 길이

가장 긴 공통 부분 수열을 찾는 과정은 다음과 같다. 가장 긴 공통 부분 수열을 찾는 과정은 경로 유사성 비교, 중간 LCS 점수 계산 및 최적화로 구성되어 있다. 첫 번째 단계인 경로 유사성 비교 과정은 다음과 같다.

Step1. 의심되는 프로그램의 경로 그래프 G_{sus} , 시작점 s , 원본 프로그램 경로 ρ 를 입력한다.

Step2. 자료구조 큐 Q 에 s 를 삽입해 BFS(Breadth First Search)를 진행하고 테이블 δ 와 배열 γ, σ 와 인덱스 r 을 0으로 초기화한다.

Step3. 큐가 비어있지 않으면 현재 노드를 Q 에서 제거한 후, 각 이웃 노드 μ 에 대해, ρ 와의 LCS 함수를 이용해 LCS점수를 계산한다.

Step4. 각 이웃 노드를 탐색하여 이웃 노드들과 ρ 와의 LCS 점수를 갱신한다.

중간 LCS 점수 계산 및 최적화 단계의 동작 과정은 다음과 같다.

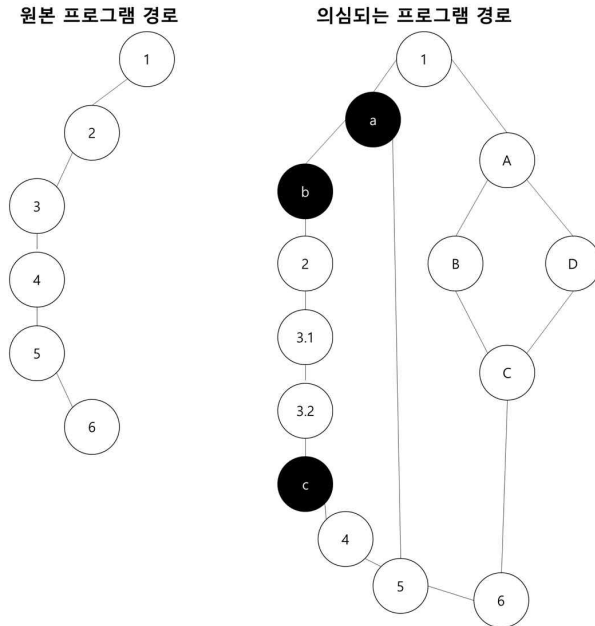
Step1. 현재 LCS 테이블 δ 에서 가장 높은 점수 $\hat{h}$ 를 찾는다.

Step2. $\hat{h}$ 가 설정한 임계값보다 높을 시 현재까지 계산된 LCS를 정제하고, 분리되거나 난독화된 기본 블록들을 병합하여 δ 를 업데이트 한다. 이를 통해 새로운 최고 LCS 점수가 계산된다.

Step3. δ 를 업데이트 후, 더 이상 최고 LCS 점수가 갱신되지 않을 경우 해당 단계를 종료한다.

LCS 함수의 동작 과정은 다음과 같다.

- Step1. LCS 테이블 $\delta(\mu, 0)$ 을 0으로 초기화 한다. 여기서 μ 는 의심되는 프로그램의 경로를 의미한다.
- Step2. 원본 프로그램 경로 ρ 의 각각의 노드 v 를 의심되는 μ 와 비교하여 LCS 점수를 계산한다. 이 때, 기본 블록 동등성 검사 함수인 $SEBB(\mu, v)$ 를 이용해 두 노드가 의미적으로 동등한 블록인지 검사한다.
- Step3. 의미적으로 동등한 함수일 경우 $\delta(\mu, v) = \delta(\text{parent}(\mu), \text{parent}(v)) + 1$ 를 통해 $\delta(\mu, v)$ 값을 갱신한다.
- Step4. 의미적으로 다른 함수일 경우 $\delta(\mu, v) = \max(\delta(\text{parent}(\mu), v), \delta(\mu, \text{parent}(v)))$ 를 통해 이전 값 중 큰 값을 갖는다.
- Step5. 모든 노드 비교가 끝나면 계산된 LCS 값을 반환한다.



<그림 III-1-2> 경로 유사도 검사 예시

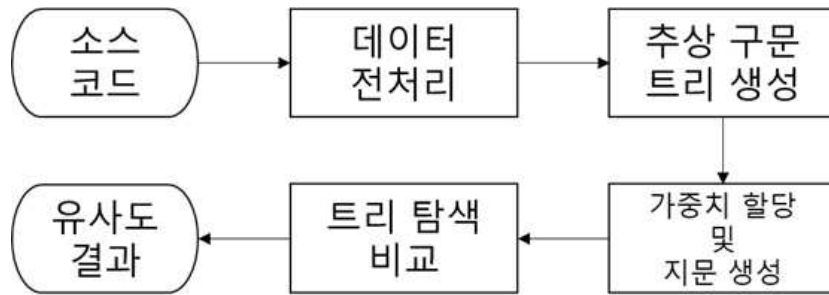
그림 <그림 III-1-2>는 원본 프로그램 경로와 의심되는 프로그램 경로를 비교하여 경로 유사도를 계산하는 그림이다. 여기서 검은색 블록은 소스코드 유사도 검사 회피를 위해 삽입된 더미 블록을 의미한다. 여기서 의심되는 프로그램 경로는 원본 프로그램 경로와 유사한 구조를 갖고 있다. 의심되는 프로그램 경로 중 1-a-b-2-3.1-3.2-4-5-6 구조가 원본 프로그램 경로와 유사한 형태를 띠고 있는 것을 볼 수 있다. 여기서 검은색 노드인 a, b, c는 유사도 검사 회피를 위해 삽입된 더미 블록이니 이를 제외하고, 3.1, 3.2는 기존 하나의 역할을 하던 블록 3을 분리하여 두 개로 나눈 것을 의미하기에 이 또한 하나의 블록으로 취급한다. 결국, 이는 원본 프로그램의 구조와 동일한 구조를 갖게 된다.

CoP 방식은 소스코드 유사도 검사 회피를 위한 변수명 변경, 함수명 변경, 소스코드 구조 변경 등 소스코드 수정에 내성을 갖는다. 또한, 소스코드를 사람이 알아볼 수 없게 변형시키는 소스코드 난독화에도 저항성을 갖는다는 장점이 존재한다. 이는 CoP 방식이 소스코드의 의미를 분석하여 유사도를 측정하므로 소스코드의 형태를 고려하지 않기 때문이다.

하지만 CoP 방식은 소스코드를 바이너리 파일로 변환한 후, 이를 다시 CFG로 변환하는 등 소스코드를 그래프로 변환하기 위해 추가적인 작업이 필요하다. 또한, CoP 방식은 소스코드 유사도 검사를 위해 소스코드를 바이너리 파일로 변환해야 한다. 이는 정적 컴파일 언어에 속하는 C, C++와 같은 언어들에는 적용할 수 있지만, 동적 컴파일 언어인 Java, C# 등에는 적용할 수 없다. Java, C#은 컴파일 시 바이너리 파일이 아닌 바이트 코드 형식으로 컴파일되어 각 언어에 종속되는 가상 머신을 통해 실행되기 때문이다.

3.1.2 추상 구문 트리 기반의 소스코드 유사도 검사

Suttichaya et al.은 추상 구문 트리 기반의 지문 정보를 활용한 소스코드 유사도 검사 기법을 제안했다[*]. 해당 방식은 기존 소스코드 유사도 검사 알고리즘의 단점인 소스코드 수정 시 정확도 감소, 성능을 극대화하기 위한 프로그래밍 언어에 종속되는 설정 등의 문제를 개선하기 위해 C 언어를 기반으로 한 추상 구문 트리 지문 기반의 소스코드 유사도 검사 기법을 제안했다. 해당 방식의 구조는 <그림 III-1-3>과 같다. 해당 방식의 동작 과정은 아래와 같다.



〈그림 III-1-3〉 Suttichaya et al. 알고리즘 구조

I. 데이터 전처리

데이터 전처리 단계는 소스코드 내 프로그램의 동작과 관련 없는 내용을 삭제하고 여러 방식으로 구현될 수 있는 기능들을 한 가지 형태로 대체하는 작업을 의미한다.

Step1. 주석, 변수명, 함수명 등 프로그램 동작 과정에 영향을 미치지 않는 값을 삭제한다.

Step2. printf()와 fprintf()와 같이 동일한 기능을 하는 서로 다른 함수들을 한 가지로 통일한다.

II. 추상 구문 트리 생성

전처리가 끝난 소스코드를 추상 구문 트리 기반의 유사도 검사를 진행하기 위한 추상 구문 트리로 변환하는 과정이다. 해당 과정에서 파서를 통해 소스코드를 구문 트리로 변환한다. 소스코드를 구문 트리로 변경하는 과정은 다음과 같다.

Step1. 소스코드를 입력해 어휘 분석을 진행한다. 어휘 분석을 통해 소스코드는 변수명, 연산자, 숫자, 예약어 등으로 구성된 토큰으로 분리된다.

Step2. 생성된 토큰을 기반으로 소스코드가 문법 규칙을 준수하는지 확인한다. 문법 규칙을 준수하는 구조를 구문 트리로 변환한다.

Step3. 구문 트리 내 프로그램의 논리와 소스코드 구조와 관계 없는 불필요한 정보를 포함한 구문 트리의 노드를 제거한다.

Step4. 위 과정을 통해 간소화된 트리가 추상 구문 트리이며, 이를 생성한 후 해당 단계를 종료한다.

III. 가중치 할당 및 지문 생성

추상 구문 트리가 생성되면 추상 구문 트리의 서브 트리에 가중치와 지문을 부여하여 이를 식별하는 과정을 진행한다. 지문과 가중치는 추상 구문 트리를 후위 순회하며 할당된다. 해당 단계에서 사용하는 파라미터와 추상 구문 트리에 가중치 및 지문을 할당하는 과정은 다음과 같다.

- $Val(*)$: 노드 *의 값
- $H(Val(*))$: $Val(*)$ 에 대한 MD5 해시 함수
- $t_1 \cdot t_2 \cdot \dots \cdot t_i$: 노드 *의 직접적인 자식 서브 트리의 해시값
- $\cdot$: 문자열 결합 연산
- $w(*)$: 노드 *의 가중치
- j : 직접 자식 노드의 수
- $x_1, x_2, \dots, x_j$: 노드 *의 자식 노드

Step1. 후위 순회 알고리즘을 통해 추상 구문 트리를 탐색한다. 후위 순회 알고리즘은 하위 계층의 노드를 시작으로 상위 계층으로 탐색을 진행하며 이를 통해 하위 계층의 구조적 특징을 상위 계층에 반영한다.

Step2. 해당 방식에서 정의하는 서브 트리의 루트 노드 $root$ 의 지문은 다음과 같이 정의된다. $F = (root) = H(H(Val(root)) \cdot t_1 \cdot t_2 \cdot \dots \cdot t_i)$ 를 통해 자식 노드의 해시값과 현재 노드 값을 결합해 새로운 지문을 생성한다.

Step3. $root$ 에 대한 가중치는 아래 식 (*)와 같이 정의된다. 여기서 가중치는 자식 노드 존재 시 자식 노드의 가중치를 모두 더한 값에 1을 더한 값이고, 자식 노드가 없을 경우 1이다.

Step4. 위와 같은 과정을 통해 각 서브 트리에 지문과 가중치를 할당한 뒤 해당 단계를 종료한다.

$$w(\text{root}) = \begin{cases} 1 + \sum_{i=1}^j w(\text{root}_i) & \text{if } j \geq 1 \\ 1 & \text{if } j = 0 \end{cases} \quad (*)$$

IV. 트리 탐색 비교

서브 트리에 지문과 가중치가 할당된 후, 할당된 지문과 가중치를 사용해 추상 구문 트리 간 중복되는 서브 트리를 탐색한다. 지문은 중복되는 서브 트리를 식별하는데 사용되며 가중치는 해당 서브 트리 간 중복되는 노드의 수를 식별하는데 사용된다. 트리 탐색 비교 단계에서 사용된 파라미터와 과정은 다음과 같다.

- $Tree_A, Tree_B$: 비교할 추상 구문 트리
- $nodequeue$: $Tree_A$ 를 BFS 알고리즘으로 탐색하기 위한 큐
- $nodeList$: $Tree_B$ 의 노드를 저장하고 있는 리스트
- $checkNode$: $Tree_A$ 에서 현재 탐색 중인 노드
- $comparedNode$: $Tree_B$ 에서 현재 비교 중인 노드
- $flag$: 현재 탐색 중인 노드 중 $Tree_B$ 의 노드와 일치하는 노드를 찾았는지에 대한 여부를 저장하는 변수
- w : 가중치
- $matched$: $Tree_A$ 와 $Tree_B$ 간 일치하는 노드의 수

Step1. $matched$ 를 0으로 초기화하고 $Tree_A$ 의 $root$ 를 $nodequeue$ 에 삽입하여 탐색을 시작한다. 또한, $Tree_B$ 의 노드를 $nodeList$ 에 저장해 향후 $Tree_A$ 의 각 노드와 비교 시 사용한다.

Step2. BFS를 통해 $Tree_A$ 를 탐색한다. $nodequeue$ 에서 하나의 노드를 꺼내 $checkNode$ 에 저장하고 $flag$ 를 False로 설정한다.

Step3. $nodeList$ 를 탐색하며 $checkNode$ 와 동일한 노드가 존재하는지 탐색한다.

Step4. 만약 $checkNode = comparedNode$ 일 경우, w 를 $matched$ 에 합산한다. 또한, $flag$ 를 True로 설정해 현재 탐색을 중지하고 다음 노드를 탐색한다.

Step5. 만약 $checkNode \neq comparedNode$ 일 경우, $checkNode$ 의 자식 노드를 $nodequeue$ 에 저장하여 탐색을 계속한다.

Step6. 위 과정을 큐가 비어있을 때까지 반복하고 최종적으로 $matched$ 를 반환한다.

V. 유사도 점수 산출

트리 탐색 비교 과정을 통해 두 추상 구문 트리 간의 일치하는 노드를 파악했으면 이를 기반으로 두 추상 구문 트리 간의 유사도를 계산한다. 유사도 점수는 0~100 사이의 값을 가지며 값이 클수록 동일한 소스코드임을 의미한다. 해당 단계에서 사용된 파라미터는 다음과 같다.

- $N(Tree_B, Tree_A)$: 참조 트리를 $Tree_A$ 로, 비교 트리를 $Tree_B$ 로 설정했을 때 일치한 노드의 수
- N_{Tree_*} : $Tree_*$ 의 전체 노드 수

두 추상 구문 트리 간의 유사도를 계산하는 수식은 식 (*)와 같다.

$$Score(Tree_A, Tree_B) = (100)(0.5) \left[\frac{N(Tree_B, Tree_A)}{N_{Tree_A}} + \frac{N(Tree_A, Tree_B)}{N_{Tree_B}} \right] \quad (*)$$

해당 방식은 소스코드의 구문 구조를 트리 형식으로 변환해 구문적 유사도 비교 시 높은 정확도를 보인다. 또한, 데이터 전처리 과정을 통해 기존 추상 구문 트리 기반의 소스코드 유사도 검사의 한계점인 변수명 변경, 함수명 변경 등으로 인한 결과 정확도 감소 문제를 해결했다.

하지만 해당 방식은 소스코드의 구문적 유사도만을 측정할 수 있다. 그렇기 때문에 소스코드의 의미적 유사성과 구조적 유사성을 측정하지 못한다. 또한, 소스코드의 의미적 유사도성을 측정할 수 없기에 소스코드 구조적 변화에 취약하다는 한계점이 존재한다.

3.1.3 소스코드 흐름도 기반의 소스코드 유사도 검사

Zhang et al.은 프로세스 마이닝 기법을 이용한 소스코드 흐름도 기반의 소스코드 유사도 검사 기법을 제안했다[*]. 해당 방식은 소스코드를 프로그램 종속성 그래프로 변환한 후, 프로그램 종속성을 일반화하여 소스코드 흐름도를 생성한다. 해당 방식은 총 4단계로 구성되어 있다. PDG 기반 중복 구문 전처리 단계, 자동 소스코드 계측 단계, 프로세스 마이닝 기반 소스코드 흐름도 생성 단계, 소스코드 흐름도 유사도 계산 단계로 구성되어 있다.

I. PDG 기반 중복 구문 전처리

PDG 기반 중복 구문 전처리는 소스코드를 PDG로 변환하는 과정이다. 해당 과정에서 최종 구문을 설정한 뒤, 최종 구문을 기반으로 프로그램 실행에 직접적인 연관이 있는 구문만을 찾는 과정이다. PDG 기반 중복 구문 전처리 단계에서 사용되는 파라미터와 과정은 아래와 같다.

- oc : Original Code, 원본 코드. 전처리를 수행할 대상 코드
- tc : Target Code, 목표 코드. 중복 구문을 제거한 최종 코드를 의미
- $G(V, E, \mu, \delta)$: PDG로 변환된 코드
- V : 구문 노드의 집합
- E : 노드 간의 종속성 엣지 집합
- μ : 구문 노드의 타입 매핑 함수
- δ : 데이터 및 제어 종속성을 나타내는 엣지 타입 매핑 함수

Step1. 원본 코드를 PDG로 변환해 $G(V, E, \mu, \delta)$ 를 생성한다. 여기서 PDG의 각 노드는 프로그램의 한 구문을 의미하며, 엣지는 구문들 간의 종속성을 의미한다.

Step2. 최종 노드 G_{ve} 와 최종 노드에 의존하는 노드 집합 V' 를 생성한다. 여기서 최종 노드란 return, output 등으로 인해 프로그램이 종료되는 시점을 의미한다.

Step3. PDG의 모든 노드를 탐색하여 각 노드의 타입을 확인한다. 현재 노드가 return 또는 output일 경우 해당 노드를 최종 구문 ve 로 설

정한다.

Step4. 최종 구문을 찾은 뒤, DFS(Depth-First Search) 알고리즘을 통해 ve 에 종속되는 모든 구문을 찾는다. 이를 통해 V' 를 구성한다.

Step5. PDG의 각 노드 중 $v \notin V'$ 인 노드들을 삭제하여 중복 구문을 제거한다. $v \notin V'$ 인 노드는 프로그램의 실행에 영향을 주지 않는 구문이므로 이는 불필요한 구문 노드이기 때문이다.

Step6. oc 에서 중복 구문을 제거한 뒤, 결과를 tc 로 갱신하여 최종적으로 프로그램의 핵심 구문만을 남긴다. 이후 tc 를 반환하고 해당 단계를 종료한다.

II. 자동 소스코드 계측

자동 소스코드 계측은 실행 로그를 삽입해 코드의 흐름을 추적하는 과정을 의미한다. 이를 위해 (Assign, Output)과 같은 계측 구문이 사용되며 해당 단계의 과정은 다음과 같다.

Step1. 실행 로그를 삽입하여 변수 선언 및 변수 할당 구문에 대한 로그를 생성한다. 변수 선언 및 할당 구문 발생 시 해당 구문 뒤에 Assign 구문을 삽입한다. 만약 블록 내 연속적인 할당 구문이 있고, 그 사이 데이터 의존성이 없으면 마지막 할당 구문 뒤 결합된 $Assign(Type_1, \dots, Type_n)$ 를 삽입하고, 데이터 의존성이 있을 경우 각 할당 구문 뒤 각각의 Assign 구문을 삽입한다.

Step2. 출력 구문 등장 시 해당 구문 뒤에 Output 계측 구문을 삽입해 해당 구문이 실행됨을 기록한다.

Step3. 동일한 유형의 계측 구문을 실행 순서에 따라 순차적으로 번호를 매긴다.

Step4. if문, 반복문 등 여러 개의 기본 블록이 존재하는 경우, 각 블록에 대해 Assign 구문을 삽입한다.

III. 자동 프로세스 마이닝 기반 소스코드 흐름도 생성

자동 프로세스 마이닝 기반 소스코드 흐름도 생성은 실행 로그를 수집하고, 이를 분석해 코드의 실행 흐름을 시각적으로 표현하는 과정이다. Assign 및 Output 구문을 활용해 코드의 동적 실행 과정을 추적한다. 또한, 휴리스틱 마

이닝 알고리즘을 이용해 반복문, 분기문과 같은 실행 흐름을 반영한 CDFD(Code Dynamic Flow Chart)를 생성한다. 자동 프로세스 마이닝 기반 소스코드 흐름도 생성 단계에서 사용되는 파라미터와 과정은 다음과 같다.

- T_f : 직접 의존성 임계값
- T_d : 의존성 임계값
- $num[]$: 두 출력 사이의 직접적인 연결 횟수를 기록하는 배열
- $d[]$: 두 출력 사이의 의존성을 나타내는 배열
- IO : 예측된 출력 집합
- V : CDFC의 노드 집합
- E : CDFC의 엣지 집합

Step1. 예측된 출력 시퀀스의 인접한 예측 출력 쌍을 순차적으로 탐색해 두 출력 사이의 직접 연결 횟수를 기록한다. 또한, 노드가 V 에 포함되지 않았다면 이를 추가한다.

Step2. 각 시퀀스의 인접한 예측 출력에 대해 $num[trace[i]][trace[i+1]]++$ 를 통해 두 출력 사이의 직접적인 연결 횟수를 증가시킨다. 또한, 노드가 V 에 포함되지 않았다면 이를 추가한다.

Step3. 예측된 출력 쌍 i_{o1}, i_{o2} 에 대해 의존성을 계산한다. 이 때 두 예측 출력이 동일한 경우, $num[i_{o1}][i_{o2}]$ 를 기반으로 $d[i_{o1}][i_{o2}]$ 를 계산한다. 이때 $num[i_{o1}][i_{o2}]$ 가 임계값 T_f 를, $d[i_{o1}][i_{o2}]$ 가 임계값 T_d 를 넘으면 엣지 (i_{o1}, i_{o2}) 를 E 에 추가한다.

Step4. 두 예측 출력이 다른 경우, num 의 차이를 기반으로 $d[i_{o1}][i_{o2}]$ 를 계산한다. 이 또한 $num[i_{o1}][i_{o2}]$ 가 임계값 T_f 를, $d[i_{o1}][i_{o2}]$ 가 임계값 T_d 를 넘으면 엣지 (i_{o1}, i_{o2}) 를 E 에 추가한다.

Step5. 생성된 $CDFC(V, E)$ 를 반환하고 해당 단계를 종료한다.

IV. 소스코드 흐름도 유사도 계산

소스코드 흐름도 유사도 계산 단계는 위 과정을 통해 생성된 두 CDFC 간의 유사성을 측정하는 단계이다. CDFC 유사도 측정을 위해 해당 기법에선 그래프 편집 거리 알고리즘을 사용한다. 그래프 편집 거리에 대한 설명은 2.3절을

참고한다. 소스코드 흐름도 유사도 계산 단계에서 사용되는 파라미터와 동작 과정은 다음과 같다.

- $CDFC_k(V_*, E_*)$: k 번째 CDFC 그래프
- V_* : k 번째 CDFC의 노드 집합, $V_* = \{v_{*1}, v_{*2}, \dots, v_{*n}\}$
- $OPEN$: 현재까지 탐색한 그래프 편집 경로를 저장하는 집합
- λ_{min} : 현재까지 계산된 최소 비용 편집 경로
- $g(\lambda)$: 시작점에서 현재 탐색 지점까지의 누적 거리값
- $h(\lambda)$: 현재 지점에서 목표 지점까지의 예상 거리
- ϵ : 공집합, 노드 삭제를 의미

Step1. $OPEN$, λ_{min} 를 각각 빈집합과 빈 값으로 초기화를 진행한다.

Step2. V_2 의 각 노드 w 에 대해 $OPEN$ 에 $v_{11} \rightarrow w$ 를 추가한다. 또한, 각 노드 w 에 대한 노드 삭제 편집 경로를 추가한다.

Step3. $OPEN$ 에 저장된 편집 경로 중 $g(\lambda) + h(\lambda)$ 가 가장 작은 경로를 선택해 이를 λ_{min} 로 설정한다.

Step4. 만약 λ_{min} 이 최소 편집 비용을 가진 경로이면 이는 편집 거리 계산이 완료된 것이므로 해당 경로를 반환하고 단계를 종료한다.

Step5. 만약 추가적인 편집 경로가 존재할 경우 $k+1$ 번째 노드와의 매칭을 위해 $v_{k+1} \rightarrow w$ 또는 노드 삭제 경로를 추가한다. 이후 위 과정을 통해 가장 작은 편집 비용을 가진 경로 λ_{min} 를 반환하고 단계를 종료한다.

Zhang et al. 방식은 루프문을 해제하는 루프 풀기, 함수를 병합/분할 하는 함수 인라이닝 및 아웃라이닝 같은 고수준의 코드 난독화 기법에 대응할 수 있다는 장점이 존재한다. 또한, 소스코드의 동적 실행 경로를 기반으로 유사도를 비교하기에 변수명 변경, 주석 변경 등 소스코드 표면을 수정해도 유사도 검사 결과에 영향을 미치지 않는다.

하지만 해당 방식은 실행 로그를 기반으로 유사도를 측정한다. 즉, 소스 코드가 정상적으로 실행되지 않으면 정확한 소스코드 흐름도 생성을 할 수 없

다. 또한, 동적 분석을 기반으로 하기 때문에 소스코드 내 구조적 변경이 발생할 경우 이를 분석하기 어렵다는 한계점이 존재한다.

3.2 k -walk 기반의 휴리스틱 알고리즘

Gaüzère et al.은 bag of walks라는 구조를 이용한 이분 매칭 기법을 이용한 그래프 편집 거리 휴리스틱 알고리즘을 제안했다[*]. 해당 방식은 기존 이분 매칭 기법의 성능 저하 문제를 해결하기 위해 bag of walks라는 구조를 활용해 이분 매칭을 진행함으로써 이전 기법보다 편집 거리 계산에 필요한 계산 비용을 감소시켰다.

해당 방식은 그래프 편집 거리 근사를 위해 k -walk라는 구조를 제안했다. k -walk란 그래프 $G=(V,E)$ 의 노드 집합 V 에 속하는 노드들로 이루어진 k 길 이만큼의 시퀀스를 의미한다. 즉, G 에 대한 그래프 탐색 과정 중 시작 노드를 기준으로 k 번의 탐색으로 구성할 수 있는 노드 시퀀스이다. 이러한 k -walk를 생성한 후 Bag of Walks라는 구조로 구성하여 이후 LSAP 기법을 이용해 그래프 편집 거리의 근사값을 도출한다.

해당 방식은 Bag of Walks 생성 단계, 비용 행렬 생성 단계, 이분 매칭 수행 단계로 구분할 수 있다. 이 중 Bag of Walks 생성 단계에서 본 제안방식을 통해 개선하고자 하는 Tottering 현상이 발생할 수 있다.

Bag of Walks 생성 단계는 그래프 편집 거리 근사값 계산을 위해 그래프 구조 파악을 위한 k -walk의 집합인 Bag of Walks를 생성하는 단계이다. 해당 과정은 그래프 G 의 노드 집합 $V=\{v_0, v_1, \dots, v_n\}$ 에 속하는 임의의 노드 v_n 을 시작 노드로 하여 k 길 이만큼의 탐색을 통해 구성할 수 있는 모든 경로를 탐색한다. 즉, k -walk는 $(\mu(v_0), \nu(v_0, v_1), \dots, \mu(v_k))$ 로 이루어진 노드 시퀀스를 의미한다. 여기서 μ, ν 는 각각 노드 라벨링 함수와 엣지 라벨링 함수를 의미한다. Bag of Walks는 이러한 k -walk들로 구성된 집합이다. Bag of Walks의 생성 과정은 다음과 같다.

Step1. 주어진 그래프 G 의 각각의 노드 v_n 에 대해 k 길 이만큼의 탐색을 통해 구성할 수 있는 모든 경로를 탐색한다. 탐색은 임의의 시작 노드 v_n 을 기준으로 v_n 과 엣지로 연결된 이웃 노드 v_{ne} 를 k 길이

만큼 탐색한다.

Step2. 탐색한 노드를 시퀀스로 저장한다. 시퀀스 $s = (\mu(v_0), \nu(v_0, v_1), \mu(v_1), \nu(v_1, v_2), \dots, \mu(v_k))$ 는 다음과 같이 구성된다.

Step3. 모든 노드를 시작 노드로 기준으로 하여 s 를 생성한 후, G 의 Bag of Walks $B_i = \{s_0, s_1, \dots, s_o\}$ 와 같이 구성된다.

즉, 해당 방식에서 Bag of Walks를 구성하기 위해 그래프 내 모든 노드를 시작 노드로 하여 k 길이의 탐색을 진행하는 것이다. 이를 예시로 설명하면 다음과 같다. 그래프 G 의 노드 집합이 $V=\{1, 2, 3, 4\}$ 이고 엣지 집합이 $E=\{(1,2), (1,3), (1,4), (2,3), (2,4), (3,4)\}$ 라 가정한다. 또한 탐색을 위한 길이는 $k=3$ 이라고 가정한다.

Step1. 시작 노드를 1로 설정하여 탐색을 시작한다. 이후, 1과 엣지로 연결된 이웃 노드인 2, 3, 4 중 인덱스가 가장 작은 2(v_1)를 다음 탐색 노드로 설정한다.

Step2. 다음 탐색 노드인 2로 이동하여 탐색을 진행한다. 이후, 2와 연결된 이웃 노드인 1, 3, 4 중 인덱스가 가장 작은 1(v_0)을 다음 탐색 노드로 설정한다.

Step3. 해당 k -walk 구성에선 위 과정을 반복하여 $s_0 = (\mu(1), \nu(1,2), \mu(2), \nu(2,1), \mu(1), \nu(1,2), \mu(2))$ 가 생성된다.

Step4. 네 번째 탐색이 종료된 후, 세 번째 탐색으로 복귀하여 노드 1의 남은 이웃 노드 3과 4에 대한 탐색을 진행한다. 이를 통해 $s_1 = (\mu(1), \nu(1,2), \mu(2), \nu(2,1), \mu(1), \nu(1,3), \mu(3))$ 과 $s_2 = (\mu(1), \nu(1,2), \mu(2), \nu(2,1), \mu(1), \nu(1,4), \mu(4))$ 가 생성된다.

Step5. 네 번째 탐색에서 1의 모든 이웃 노드를 탐색했을 경우 세 번째 탐색으로 복귀하여 Step4와 같이 남은 이웃 노드를 통해 갈 수 있는 모든 경로를 탐색하여 최종적으로 B_i 를 생성한다.

위 과정을 V 에 속하는 모든 노드 v_n 을 시작 노드로 설정하여 진행해 최종적으로 $B = \{B_1, B_2, \dots, B_i\}$ 를 생성한다.

해당 방식은 위와 같은 과정으로 생성된 k -walk를 기반으로 이후 LSAP 기

제 4장 요구사항 분석

반의 그래프 편집 거리 근사 알고리즘을 수행한다. 하지만 해당 방식의 k -walk 생성 방식은 Tottering 현상이 발생한다는 문제점이 존재한다. Tottering 현상은 그래프 탐색 시 두 노드를 번갈아가며 탐색하는 현상이므로써 자세한 설명은 2.5절 Tottering 현상을 참고한다.

해당 방식은 위의 Bag of Walks 생성 과정 예시에서 Tottering 현상이 발생한 것을 확인할 수 있다. 예시 중 Step3. 과정에서 생성된 s_0 의 경우 $s_0 = (\mu(1), \nu(1,2), \mu(2), \nu(2,1), \mu(1), \nu(1,2), \mu(2))$ 와 같이 시퀀스가 생성되었다. 이를 노드 탐색 순서로 표현 시 (1-2-1-2)와 같이 표현할 수 있다. 여기서 (1-2-1), (2-1-2)와 같이 1과 2 두 노드만을 번갈아가며 탐색을 진행함으로써 Tottering 현상이 발생한다.

이로 인해 이후 과정에서 진행되는 LSAPF 기반의 그래프 편집 거리 근사 알고리즘 과정에서 근사값 정확도 감소, 알고리즘 실행 시간 증가 등 다양한 문제가 발생한다. 이는 해당 방식이 그래프 탐색 과정 중 노드의 중복 방문을 고려하지 않기 때문에 발생하는 문제이다.

그래프 탐색 과정 중 이러한 Tottering 현상을 방지하기 위해선 노드의 중복 방문을 식별하고 이를 방지해야 한다. 노드의 중복 방문을 식별하여 Tottering 현상이 발생한 k -walk의 생성을 방지하면 Tottering 현상이 발생한 k -walk로 인해 야기되었던 문제들을 해결할 수 있다.

따라서 본 논문에선 기존 k -walk 기반의 휴리스틱 알고리즘 과정 중 k -walk 생성 과정에서 발생하는 Tottering 현상을 방지하는 새로운 k -walk 생성 알고리즘을 제안한다. 이를 통해 불필요한 k -walk 생성 방지, 근사값 정확도 향상, 실행 시간 감소 등의 효과를 볼 수 있다. 본 제안방식의 Tottering 현상을 방지하는 k -walk 생성 알고리즘은 5.2절에서 자세히 설명한다.

본 장에서는 효율적인 OSS 유사도 검사를 위한 PDG 및 k -walk 기반의 소스코드 유사도 검사를 위해 필요한 요구사항을 설명한다. 본 제안방식은 소스코드를 PDG 그래프로 변환하여 소스코드 유사도 검사를 진행하는 PDG 기반의 소스코드 유사도 검사 기법과 기존 k -walk 생성 기법에서 Tottering 현상을 방지하는 새로운 k -walk 생성 기법이다. 각 요구사항은 이하 절에서 설명한다.

4.1 Tottering 현상을 방지하는 k -walk 생성 기법

Tottering 현상을 방지하는 k -walk 생성 기법은 기존 k -walk 기반의 휴리스틱 알고리즘 중 k -walk 생성 과정에서 발생하는 Tottering 현상을 개선하기 위한 새로운 k -walk 생성 기법이다. 이에 대한 요구사항은 다음과 같다.

4.1.1 Tottering 현상 개선

기존 k -walk 기반의 휴리스틱 알고리즘은 k -walk 생성 과정에서 중복 방문이 발생하여 Tottering 현상이 발생한다. Tottering 현상으로 인해 불필요한 k -walk 생성, 실행 시간 증가, 근사값 정확도 감소 등 다양한 문제가 발생하며 이는 곧 효율성, 정확성 감소 등의 문제로 이어진다. 따라서 이를 방지하기 위해 그래프 탐색 중 발생하는 Tottering 현상을 방지해야 한다.

Tottering 현상을 방지하기 위한 방법 중 하나는 그래프 탐색 과정 중 중복 방문을 탐지하여 Tottering 현상을 방지하는 것이다. 중복 방문을 방지하기 위해 그래프의 노드를 탐색하며 현재 노드를 기준으로 이전 탐색 노드와 현재 탐색 노드를 비교하면 효과적으로 Tottering 현상을 방지할 수 있다. 즉, 현재 탐색 노드를 v_c 라 하고, 이전 탐색 노드를 v_p 라고 했을 때, $v_c = v_p$ 일 경우 Tottering 현상이 발생했음을 알 수 있다. 중복 방문 검사를 통해 k -walk 기반의 휴리스틱 알고리즘에서 발생한 Tottering 현상을 방지할 수 있어야 한다.

4.1.2 실행 시간 개선

4.1.1절에서 언급했던 것과 같이 기존 방식은 Tottering 현상으로 인해 불필요한 탐색이 발생하고 그로 인해 생성되는 비용 행렬 또한 커지게 된다. 이는 LSAPE-GED와 같은 최적합 할당 문제에서 큰 문제로 이어진다. 비용 행렬이 커짐으로써 알고리즘의 실행 시간 또한 크게 증가하게 된다. 이때 LSAPE-GED의 시간 복잡도는 선형적으로 증가하기 때문에 비용 행렬이 커질수록 알고리즘의 실행 시간 또한 크게 증가한다. 따라서 기존 k -walk 기반의 휴리스틱 알고리즘의 실행 시간을 개선해야 한다.

4.1.3 정확도 개선

기존 k -walk 기반의 휴리스틱 알고리즘은 Tottering 현상으로 인해 알고리즘의 정확도가 감소한다. 즉, 불필요한 탐색으로 인해 구축해야 할 비용 행렬이 커지고, 이는 즉 휴리스틱 알고리즘의 하한이 부정확해지는 결과를 초래한다. LSAPE-GED는 모든 노드 매칭을 확인한 후, 각 행에서 최소 비용을 갖는 매칭을 선택해 그래프 편집 거리를 계산한다. 만약 Tottering 현상이 발생하지 않은 그래프 탐색 결과로 만들어진 비용 행렬이 5×5 행렬이라고 가정하면, Tottering 현상이 발생한 그래프 탐색 결과로 만들어진 비용 행렬은 5×5 이상의 행렬이 생성된다. 즉, Tottering 현상이 발생한 비용 행렬은 7×7 , 10×10 등 정상적인 비용 행렬보다 크게 생성된다.

이때 각 행의 최소 비용이 1이라고 가정하면 정상적인 5×5 행렬의 최소 비용은 5로 계산된다. 하지만 Tottering 현상이 발생한 비용 행렬은 7×7 행렬일 경우 최소 비용은 7, 10×10 행렬의 경우 10 등 정상적인 행렬의 최소 비용과 차이가 발생한다. 이로 인해 휴리스틱 알고리즘으로 계산된 결과의 정확도에 영향을 미칠 수 있다. 효율적인 알고리즘을 위해 기존 k -walk 기반의 휴리스틱 알고리즘에서 발생하는 정확도 감소 문제를 개선해야 한다.

4.2 PDG 기반의 소스코드 유사도 검사 기법

PDG 기반의 소스코드 유사도 검사는 기존 텍스트 기반의 소스코드 유사도 검사의 한계를 극복하고 효과적인 소스코드 유사도 검사를 위해 제안된 기법이다. 이에 대한 요구사항은 다음과 같다.

4.2.1 소스코드 기반의 PDG 생성

PDG 기반의 소스코드 유사도 검사는 소스코드를 PDG 그래프로 변환하여 변환된 그래프를 기반으로 소스코드 유사도 검사를 진행하는 기법이다. 이를 위해 반드시 소스코드를 기반으로 한 PDG가 필요하다. 생성된 PDG는 소스코드의 데이터 종속성과 제어 종속성을 모두 반영해야 한다.

데이터 종속성이란 소스코드 내 변수 또는 데이터의 흐름과 관련된 종속성을 의미한다. 이는 명령문이 다른 명령문에 의존하는 경우, 의존 관계가 변수를 사용하는 방식일 때 속한다.

제어 종속성은 조건문 또는 분기문 등으로 인해 프로그램 실행 흐름이 결정될 때 발생한다. 즉, if-else문 또는 for, while과 같은 반복문 등 이러한 제어 구조에 따라 프로그램의 실행이 달라지는 것을 의미한다.

추가적으로 PDG를 구성하는 노드와 엣지는 다음과 같은 속성을 갖는다. PDG 내 노드들은 소스코드 내 함수들을 노드로 표현한 것이며, 엣지는 각 함수 간의 종속성을 엣지로 표현한 것이다. 이를 통해 소스코드 내 함수 간의 종속 관계를 그래프로 표현한 PDG를 생성할 수 있다. 본 제안방식에선 생성된 PDG는 dot 파일로 구성된다. dot 파일의 구성은 <그림 IV-2-1-1>과 같다. dot 파일 내에는 소스코드 내 함수와 함수들 간의 종속 정보를 포함한다.

```
digraph G{
  graph [rankdir=TB];
  subgraph cluster_G {
    graph [style="filled,rounded", fillcolor="#80808018", label=""];
    pyan__sphinx [label="pyan.sphinx", style="filled", fillcolor="#ffffffb2", fontcolor="#000000", group="0"];
    pyan__sphinx_CallgraphXDirective [label="CallgraphDirective", style="filled", fillcolor="#ffffffb2", fontcolor="#000000", group="0"];
    pyan__sphinx_direction_spec [label="direction_spec", style="filled", fillcolor="#ffffffb2", fontcolor="#000000", group="0"];
    pyan__sphinx_setup [label="setup", style="filled", fillcolor="#ffffffb2", fontcolor="#000000", group="0"];
    pyan__sphinx_CallgraphXDirective_run [label="run", style="filled", fillcolor="#ffffffb2", fontcolor="#000000", group="0"];
    pyan__sphinx_CallgraphXDirective_run_create_link [label="create_link", style="filled", fillcolor="#ffffffb2", fontcolor="#000000", group="0"];
  }
  pyan__sphinx_CallgraphXDirective --> pyan__sphinx_direction_spec [style="solid", color="#000000"];
  pyan__sphinx_CallgraphXDirective_run --> pyan__sphinx_CallgraphXDirective_run_create_link [style="solid", color="#000000"];
  pyan__sphinx_setup --> pyan__sphinx_CallgraphXDirective [style="solid", color="#000000"];
}
```

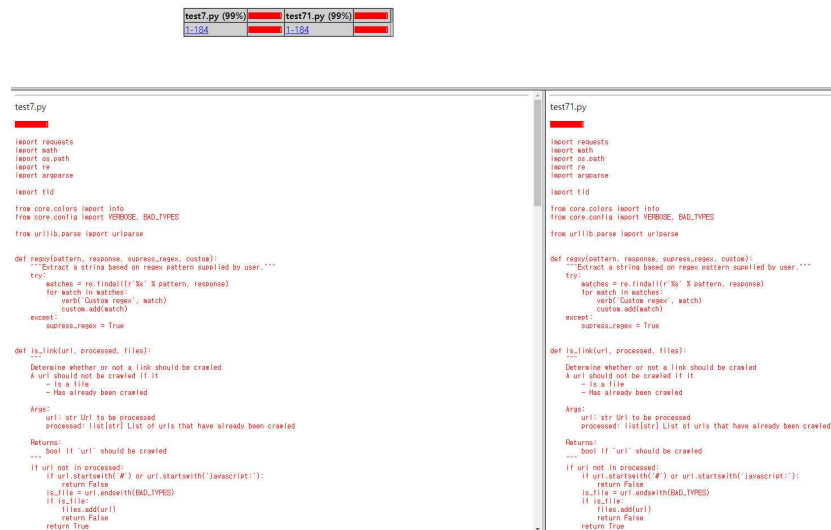
<그림 IV-2-1-1> dot 파일 예시

4.2.2 신뢰성

텍스트 기반의 소스코드 유사도 검사의 한계점 중 하나는 낮은 신뢰성이다. 텍스트 기반의 소스코드 유사도 검사는 동일한 소스코드를 비교해도 유사도 결과가 98%, 99% 등 100%의 유사도를 검출하지 못한다. <그림 IV-2-2-1>은 대표적인 텍스트 기반의 소스코드 유사도 검사 도구인 MOSS(Measure Of Software Similarity)를 이용해 동일한 소스코드를 비교한 결과이다. 두 소스코드의 내용은 모두 동일하며 파일의 이름만 변경한 상태이다.

이는 물론 높은 수준의 유사도를 보이고 있지만 오탐 및 미탐의 가능성을 포함하고 있다. 오탐으로 인해 높은 유사도를 보였지만 실제 표절이 아닌 경우, 불필요한 분쟁이 발생하고 유사도 검사 시스템의 신뢰도가 저하될 수 있다. 또는 미탐으로 인해 실제로 표절이지만 코드 리팩토링 등을 통해 유사도 검사를 회피할 수 있다. 즉, 유사도 결과가 99%일지라도 두 소스코드가 동일한 소스코드임을 신뢰할 수 없다.

이러한 문제를 방지하기 위해 두 소스코드가 동일한 소스코드일 경우, 100%의 유사도를 검출할 수 있어야 한다. MOSS와 같은 텍스트 기반의 소스코드



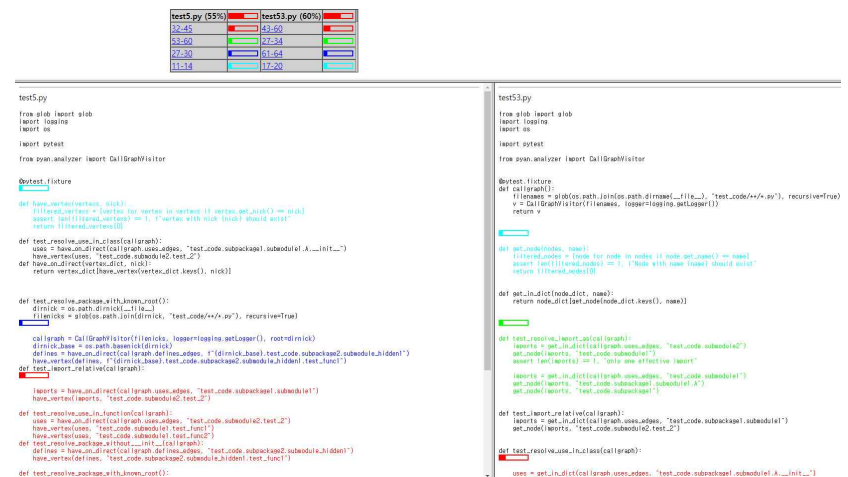
<그림 IV-2-2-1> MOSS 동일 소스코드 비교 결과

유사도 검사가 동일한 소스코드 내용을 동일하지 않다고 판단하는 이유는 다음과 같다. 소스코드 유사도 검사를 위하여 블록으로 분할 후 해시화하는 과정 중 블록 분할 과정에서 육안으로 구분하기 형식적 요소로 인해 블록 분할이 다르게 되어 이와 같은 문제가 발생한다.

4.2.3 코드 변형 대응

텍스트 기반의 소스코드 유사도 검사는 변수명 변경, 함수명 변경, 소스코드 구조 변경, 주석 삽입 및 삭제, 공백 삽입 및 삭제 등 다양한 요인들이 유사도 검사 결과에 영향을 준다. 특히, 소스코드의 함수 위치, 변수 위치 등을 변경하는 소스코드 구조 변경은 기존 텍스트 기반의 소스코드 유사도 검사 결과에 큰 영향을 미쳐 심각한 유사도 하락을 유발한다. <그림 IV-2-3-1>은 MOSS에서 원본 소스코드와 구조를 변경한 소스코드의 유사도 검사를 결과이다.

이처럼 정확한 소스코드 유사도 검사 결과를 제공하기 위해 변수명 변경, 함수명 변경, 소스코드 구조 변경, 그 외 다양한 소스코드 수정 등에 영향을 받지 않는 소스코드 유사도 검사 기법을 제공해야 한다.



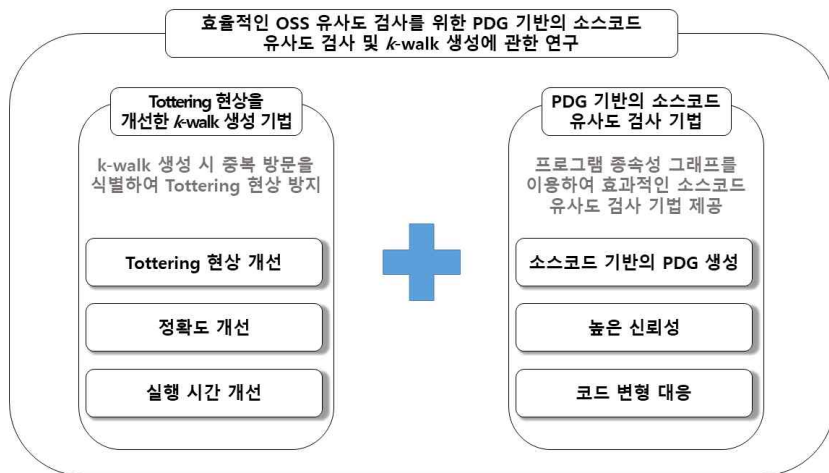
<그림 IV-2-3-1> MOSS 변형 소스코드 비교 결과

제 5장 제안방식

본 장에서는 본 논문에서 제안하고자 하는 제안방식들에 대해 기술한다. 본 논문에서 제안하는 제안방식은 총 두가지로 첫 번째 제안방식은 PDG 기반의 소스코드 유사도 검사 기법이다. 해당 방식은 기존 의미론 기반, 추상 구문 트리 기반, 소스코드 흐름도 기반의 소스코드 유사도 검사 알고리즘의 한계점을 개선하기 위해 제안하는 기법이다.

두 번째 제안방식은 Tottering 현상을 방지하는 k -walk 생성 기법이다. 해당 방식은 기존 k -walk 기반의 휴리스틱 알고리즘에서 Bag of Walks 생성 시 발생했던 Tottering 현상을 방지하기 위해 제안하는 기법이다.

그림 <V-1>은 본 제안방식의 목표이다. 각 기법에 대한 자세한 설명은 이하 절에서 설명한다.



<그림 V-1> 제안방식 목표

5.1 Tottering 현상을 방지하는 k -walk 생성 기법

본 절에서는 기존 k -walk 기반의 휴리스틱 알고리즘에서 발생했던 Tottering 현상을 방지하는 새로운 k -walk 생성 기법을 제안한다. 기존 k -walk 기반의 휴리스틱 알고리즘은 k -walk 생성 시 중복 방문을 고려하지 않아 직전에 방문했던 노드를 재방문 할 수 있다. 이로 인해 그래프 구조 파악에 불필요한 k -walk가 생성된다. 불필요한 k -walk는 (1-2-1-2), (1-2-3-2)와 같으며, 이러한 k -walk는 최종적인 알고리즘 실행 시간을 감소시키며, 결과를 통해 계산된 근사값의 정확도 또한 저하시킨다.

따라서 본 제안방식은 기존 k -walk 기반의 휴리스틱 알고리즘 중 k -walk 생성 과정을 개선하여 Tottering 현상을 방지하고자 한다. 본 제안방식은 중복 방문 식별을 위해 자료구조인 큐를 이용해 노드의 중복 방문을 식별한다. 그래프 탐색 시 탐색한 노드 정보를 큐에 저장하고, 이후 탐색 진행 시 현재 탐색 노드와 큐에 저장된 노드 정보를 비교하여 Tottering 현상이 발생했는지 확인한다.

본 제안방식은 완전 그래프 $G = \{V_G, E_G, \mu_G, \nu_G\}$ 를 가정한다. V 는 $V = \{v_0, v_1, \dots, v_n\}$ 으로 이루어진 노드 집합이며 E 는 $E = \{(v_0, v_1), (v_0, v_1), \dots, (v_{n-1}, v_n)\}$ 로 이루어진 엣지 집합을 의미한다. 여기서 G 는 n 개의 노드를 가질 경우 $n(n-1)$ 개의 엣지를 가진다. 알고리즘 *는 각각의 노드에 식별자를 부여하는 과정을 보여준다. DFS 알고리즘을 이용해 노드를 탐색하며, 방문하는 노드들에 순서대로 식별자를 부여한다.

본 제안방식에서 사용하는 파라미터는 다음과 같다.

- G : 임의의 완전 그래프
- V_* : 그래프 *의 노드 집합
- E_* : 그래프 *의 엣지 집합
- v_{st} : 시작 노드
- v_{cu} : 현재 탐색 노드
- v_{ne} : 현재 탐색 노드의 이웃 노드
- $NE_{v_{cu}}$: 현재 탐색 노드의 이웃 노드 집합

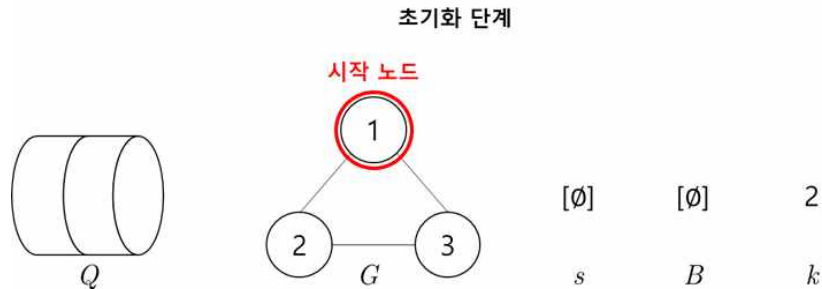
- k : 탐색 거리
- B : 생성된 모든 k -walk의 집합
- B_n : 노드 v_n 에서 생성된 k -walk의 집합
- s : 노드 시퀀스
- Q : 탐색한 노드 정보를 저장하는 크기가 2인 자료구조 큐
- $Q.enq(v_n)$: Q 에 노드 v_n 를 삽입하는 함수
- $Q.deq(q_i)$: Q 의 i 번째 원소를 출력하는 함수

I. 초기화 단계

본 제안방식은 k -walk 생성 중 Tottering 현상을 방지하기 위해 중복 방문을 탐지한다. 중복 방문 탐지를 위해 본 제안방식에선 큐를 사용한다. 큐에 탐색한 노드 정보를 저장하고, 다음 노드 탐색 시 큐에 저장된 노드와 현재 탐색 노드를 비교하여 효과적으로 중복 방문을 방지할 수 있다. 본 제안방식은 초기화 단계, 그래프 탐색 단계로 구성되어 있다. 그림 <그림 V-1-1>는 본 제안방식의 초기화 단계를 그림으로 나타낸 그림이다. 여기서 $G = V, E$ 이며, $V = \{1, 2, 3\}$ 이고 $E = \{(1, 2), (1, 3), (2, 3)\}$, $k=2$ 이다. 제안방식의 초기화 단계 과정은 다음과 같다.

Step1. 그래프 G , 탐색 거리 k 를 입력으로 하여 그래프 탐색을 시작한다.

Step2. 시작 노드 v_{st} 을 설정한다. 시작노드는 V_G 에 속하는 노드 중, 현재 탐색하지 않았으며 인덱스가 가장 작은 노드를 우선 탐색한다.



<그림 V-1-1> 제안방식 초기화 단계 예시

Step3. k -walk 생성을 위해 B 와 Q 를 $B \leftarrow \phi$, $Q \leftarrow \phi$ 와 같이 초기화한다.

위 과정을 통해 탐색할 그래프를 설정하며, 그래프의 탐색 거리, 시작노드를 설정한다. 추가적으로 향후 그래프 탐색을 통해 생성된 k -walk 및 노드 정보를 저장할 B 와 Q 를 초기화한다.

II. $|Q| < 2$ 탐색 단계

초기화 과정이 종료되면 이후 그래프 탐색 단계를 진행한다. 그래프 탐색 단계는 Q 의 길이인 $|Q|$ 가 2일 때와 2 미만일 때로 구분된다. 여기서 $|Q| < 2$ 란 Q 내의 원소가 0개 또는 1개인 것을 의미한다. <그림 V-1-2>는 본 제안방식의 $|Q| < 2$ 일 때의 탐색 과정을 나타낸 그림이다. $|Q|$ 가 2 미만일 때의 k -walk 생성 과정은 다음과 같다.

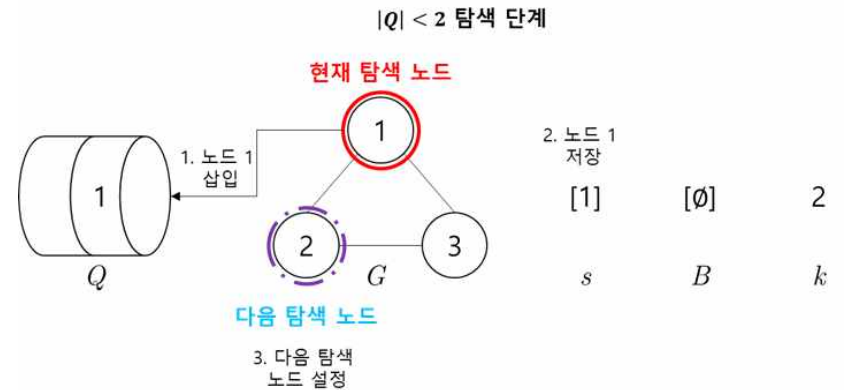
Step1. 현재 탐색 노드 v_{cu} 를 $Q.enq(v_{cu})$ 를 통해 저장한다.

Step2. 현재 탐색 노드를 시퀀스 s_o 에 $s_o \leftarrow s \cup v_{cu}$ 와 같이 저장한다.

Step3. $k=0$ 일 경우, 현재 생성된 s_o 를 B_o 에 저장한 뒤 탐색을 종료한다.

Step4. $k \neq 0$ 일 경우, v_{cu} 의 이웃 노드 v_{ne} 중 인덱스가 가장 작은 노드를 탐색 노드 v_{cu} 로 설정한다.

Step5. 탐색 노드 v_{cu} 와 $k-1$ 값을 입력해 재귀적으로 탐색을 진행한다.



<그림 V-1-2> 제안방식 $|Q| < 2$ 탐색 단계 예시

Algorithm 1 Adv_GenWalk1

Input: Graph G , node $v_{cu} \in V$, Length of the traversal k

Output: A set of walks B

```

 $B \leftarrow \phi, Q \leftarrow \phi$ 
1: if  $|Q| = 2$  then:
2:   Adv_GenWalk2( $G, v_{cu}, k, Q, B$ )
3: end if
4:  $Q.enq(v_{cu})$ 
5:  $s \leftarrow s \cup v_{cu}$ 
6: if  $k = 0$  then
7:   return  $s$ 
8: end if
9: for  $v_{ne}$  in  $NE_{v_{cu}}$  do
10:  Adv_GenWalk1( $G, v_{ne}, k - 1$ )
11:   $B \leftarrow B \cup s$ 
12: end for
13: return  $B$ 
  
```

알고리즘 1은 위 $|Q| < 2$ 일 때 본 제안방식의 동작과정을 나타낸 알고리즘이다. 알고리즘 시작 시 $|Q|$ 를 검사한다. 이는 재귀적으로 탐색하는 해당 알고리즘의 특성 상 Q 의 길이를 확인하여 Tottering 검사가 필요한지 확인하기 위한 과정이다. 알고리즘 수행 중 만약 $|Q| = 2$ 를 만족할 경우, Adv_GenWalk2를 수행하여 $|Q| = 2$ 일 때의 k -walk 생성 과정을 진행한다.

만약 $|Q| = 2$ 를 만족하지 않고 $|Q| < 2$ 를 만족할 경우, 현재 탐색 노드 v_{cu} 를 Q 에 삽입한다. 이는 향후, Tottering 현상 발생 가능성이 존재하는 $|Q| = 2$ 를 만족하는 탐색 진행 시, Tottering 현상을 검사하기 위함이다. 이후 현재 시퀀스 s 에 v_{cu} 를 저장하여 k -walk를 생성 과정을 진행한다. 현재까지의 과정이 1번째 줄부터 5번째 줄의 과정이다.

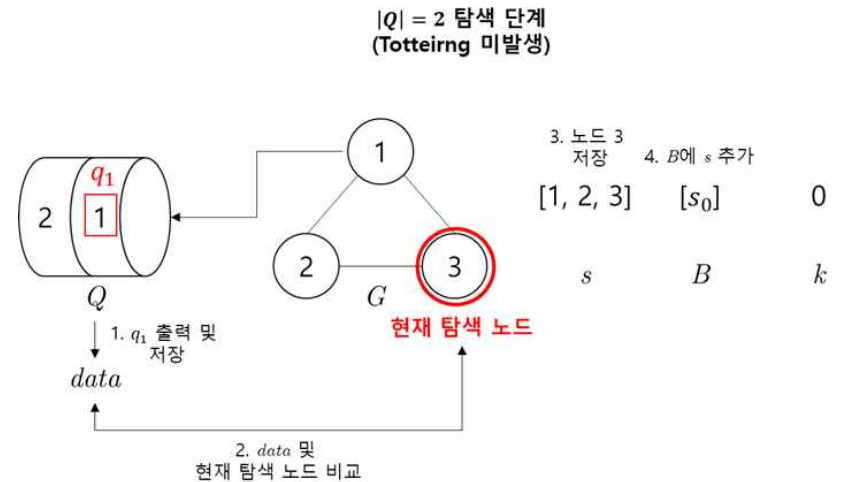
이후, 탐색 과정 중 $k = 0$ 을 만족하면 현재까지 생성된 s 를 반환한다. $k = 0$ 은 시작 노드를 기준으로 주어진 탐색 거리만큼의 탐색을 모두 진행했다는 것을 의미한다. 이 경우 정상적인 k -walk가 생성된 것을 의미한다. 따라서 현재 생성된 s 를 반환하여 11번째 줄의 과정과 같이 k -walk 집합 B 에 생성된 s 를 저장한다.

만약 $k \neq 0$ 일 경우, 즉 k 가 1 이상일 경우엔 v_{cu} 의 이웃 노드 집합 중 인덱스가 가장 작은 노드를 이웃 노드 v_{ne} 로 선정하여 Adv_GenWalk1를 수행한다.

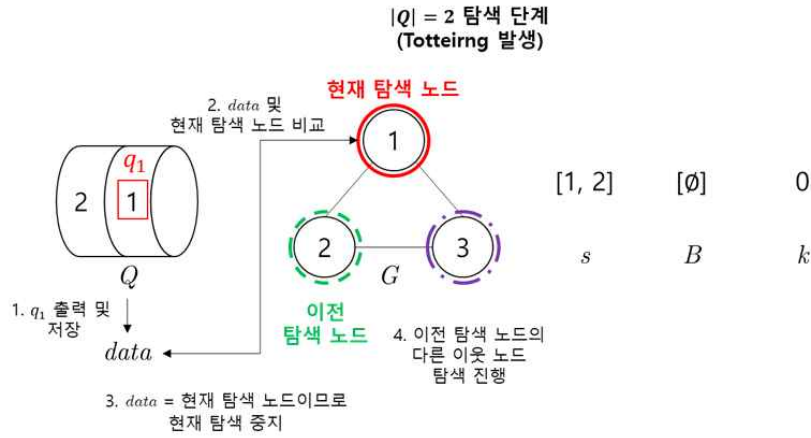
이 때 입력은 그래프 G , 탐색 거리 k , 그리고 이전 과정에서 설정했던 이웃 노드 v_{ne} 를 입력으로 한다. 이를 통해 현재 탐색 노드를 탐색 후, 이웃 노드의 탐색을 진행할 수 있다.

해당 과정을 시작 노드를 기준으로 모든 노드를 탐색하여 생성할 수 있는 k -walk를 생성할 수 있다. 단, 탐색 진행 중 $|Q| = 2$ 를 만족하게 될 경우 Adv_GenWalk2를 수행하여 탐색 과정 진행 중 Tottering 현상을 검사한다. 즉, Adv_GenWalk1은 시작 노드 v_{st} 를 기준으로 최대 2번의 탐색을 진행하여 생성할 수 있는 모든 k -walk를 생성하는 과정이다.

이와 같은 과정을 통해 $|Q| < 2$ 일 때의 탐색을 진행할 수 있다. 이는 시작 노드를 기준으로 탐색이 한 번도 일어나지 않았거나, 한 번의 탐색만이 발생했을 때의 k -walk 생성 과정을 의미한다. 이 경우 Tottering 현상이 발생할 수 없다. Tottering 현상은 두 노드를 번갈아가며 탐색하기에 발생하는 현상이다. 즉, Tottering 현상이 발생하기 위해선 최소 2번 이상의 탐색이 발생해야 한다는 것을 의미한다. 그렇기 때문에 본 제안방식은 $|Q| < 2$ 를 만족할 경우와 $|Q| = 2$ 를 만족할 경우를 구분하여 동작한다.



〈그림 V-1-3〉 제안방식 $|Q| = 2$ 탐색 단계 예시(Tottering 미발생)



〈그림 V-1-4〉 제안방식 $|Q|=2$ 탐색 단계 예시(Tottering 발생)

III. $|Q|=2$ 탐색 단계

$|Q| < 2$ 를 만족할 때의 동작 과정을 반복적으로 수행하여 $|Q|=2$ 를 만족하고, $|Q|=2$ 만족 시 Tottering 현상이 발생할 수 있는 최소 조건을 만족한 것이므로 이후 탐색에선 Q 를 이용하여 노드의 중복 방문을 검사한다. $|Q|=2$ 탐색 단계의 그림은 〈그림 V-1-3〉과 〈그림 V-1-4〉이다. 〈그림 V-1-3〉은 $|Q|=2$ 탐색 단계에서 Tottering 현상이 발생하지 않았을 때의 그림이며 〈그림 V-1-4〉는 Tottering 현상이 발생했을 때의 그림이다. $|Q|=2$ 를 만족할 때의 k -walk 생성 과정은 다음과 같다.

- Step1. Q 에 저장된 첫 번째 원소 q_1 를 $Q.deg(q_1)$ 을 통해 출력한다.
- Step2. 출력된 q_1 과 현재 탐색 노드 v_{cu} 을 비교한다. 만약 $q_1 = v_{cu}$ 일 경우, Tottering 현상이 발생했으므로 현재 탐색을 종료하고, 이전 탐색 노드의 또 다른 이웃 노드 v_{ne} 를 선택하여 다음 탐색을 진행한다.
- Step3. 만약 $q_1 \neq v_{cu}$ 일 경우, 이는 Tottering 현상이 발생하지 않은 정상적인 탐색을 의미한다. 현재 탐색 노드를 시퀀스 s_o 에 $s_o \leftarrow s \cup v_{cu}$ 와 같이 저장한다.
- Step4. $k \neq 0$ 일 경우, v_{cu} 의 이웃 노드 v_{ne} 중 인덱스가 가장 작은 노드를 탐색 노드 v_{cu} 로 설정한다.

Step5. 탐색 노드 v_{cu} 와 $k-1$ 값을 입력으로 하여 재귀적으로 탐색한다.

Step6. 만약 탐색 중 $k=0$ 을 만족할 경우, 현재까지 생성된 s_o 를 B_o 에 저장한 뒤 탐색을 종료한다.

알고리즘 2는 위 $|Q|=2$ 일 때의 본 제안방식의 동작 과정을 나타낸 알고리즘이다. *Adv_GenWalk2*는 이전 *Adv_GenWalk1*을 수행하는 도중 $|Q|=2$ 를 만족하여 수행되는 알고리즘이다. 그렇기 때문에 *Adv_GenWalk1*의 입력에 더불어 Q 와 B 를 추가적으로 입력받는다. 이는 기존 *Adv_GenWalk1*에서 생성됐던 k -walk 생성 과정을 이어 남은 탐색을 진행하기 위한 알고리즘이기 때문이다.

Adv_GenWalk2 알고리즘 시작 시 현재 Q 에 저장된 첫 번째 원소 q_1 를 출력해 $data$ 변수에 저장한다. 이는 $|Q|=2$ 를 만족하여 Tottering이 발생할 수 있는 상황이기 때문에 Tottering 현상을 검사하기 위함이다. 이후 $data$ 와 현재 탐색 노드 v_{cu} 를 비교한다. 이 때 두 값이 동일할 경우, 현재 탐색을 종료하고 이전 탐색의 또 다른 이웃 노드를 선정하여 다음 탐색을 진행한다. 만약 두 값이 다를 경우 v_{cu} 를 Q 에 삽입하고 s 에 저장하여 k -walk 생성 과정을 진행한다.

정상적으로 노드 탐색을 진행했으면 현재 탐색 노드와 엮지로 직접적으로 연결된 이웃 노드 중, 인덱스가 가장 작은 노드를 선택하여 이웃 노드 v_{ne} 로 설정한다. 이후, v_{ne} 를 현재 탐색 노드를 하여 위 과정을 반복한다. 이를 통해

Algorithm 2 Adv_GenWalk2

Input: Graph G , node $v_{cu} \in V$, Length of the traversal k , Queue Q , A set of walks B

Output: A set of walks B

```

1:  $data \leftarrow Q.deg()$ 
2: if  $v_{cu} = data$  then
3:   continue
4: end if
5:  $Q.enq(v_{cu})$ 
6:  $s \leftarrow s \cup v_{cu}$ 
7: if  $k = 0$  then
8:   return  $s$ 
9: end if
10: for  $v_{ne}$  in  $NE_{v_{cu}}$  do
11:    $Adv\_GenWalk2(G, v_{ne}, k - 1)$ 
12:    $B \leftarrow B \cup s$ 
13: end for
14: return  $B$ 

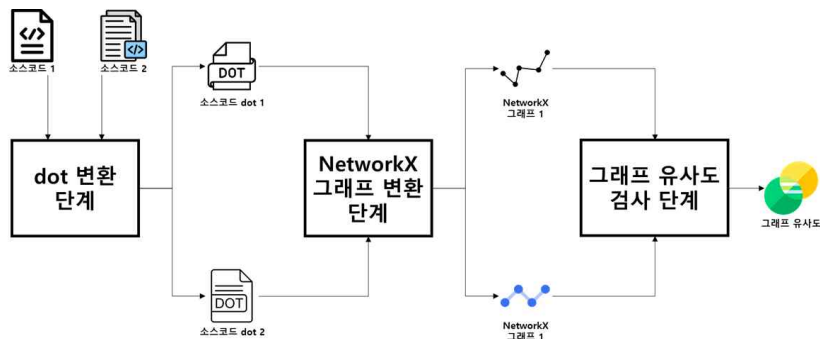
```


시작 노드 v_{st} 를 기준으로 k 번의 탐색을 통하여 생성할 수 있는 모든 k -walk를 생성할 수 있다. 이 때, *Adv_GenWalk2*를 통해 Tottering 현상이 발생한 k -walk의 생성은 효과적으로 방지할 수 있다.

본 제안방식은 기존 *GenWalk*와 달리 자료구조 Q 를 이용하여 중복 방문을 관리한다. $|Q|=2$ 만족 시 Q 에 저장된 원소를 출력해 현재 탐색 노드와 비교하고, 두 값이 동일할 경우, 해당 탐색을 중지하고 이전 탐색 노드의 다른 이웃 노드를 대상으로 탐색을 진행한다. 따라서 본 제안방식은 기존 방식의 k -walk 생성 중 발생했던 Tottering 현상을 방지할 수 있고, Tottering 현상이 발생한 k -walk의 생성을 효과적으로 방지할 수 있다. 이를 통해 불필요한 k -walk 생성 방지, k -walk 생성 시간 감소, LSAPE-GED 근사값 정확도 향상 등의 효과를 제공할 수 있다.

5.2 PDG 기반의 소스코드 유사도 검사 기법

본 절에서는 OSS 환경에서 효율적인 소스코드 유사도 검사를 실행하기 위한 PDG 기반의 소스코드 유사도 검사 기법을 제안한다. 기존 의미론 기반의 소스코드 유사도 검사는 유사도 검사를 위해 소스코드를 바이너리 파일로 변환해야했다. 이는 동적 컴파일 언어에 적용할 수 없어 적용 범위에 한계점이 존재한다. 추상 구문 트리 기반의 소스코드 유사도 검사는 소스코드의 의미적 유사성과 구조적 유사성을 측정할 수 없다는 한계점이 존재한다, 소스코드 호



<그림 V-2-1> PDG 기반의 소스코드 유사도 검사 기법 시나리오

름도 기반의 소스코드 유사도 검사는 동적 분석을 기반으로 하기 때문에 구조적 변경을 식별하기 힘들다. 또한, 기존엔 소스코드를 PDG로 변환하여 소스코드 유사도 검사를 제공하는 기법 또는 도구가 부족한 실정이다.

따라서 본 제안방식은 텍스트 기반의 소스코드 유사도 검사의 한계점인 낮은 신뢰도, 코드 변형 대응에 더불어 기존 그래프 기반의 소스코드 유사도 검사 기법들의 한계점을 극복한 기법을 제안한다. 추가적으로 소스코드를 PDG로 시각화하여 개발자가 이를 확인해 소스코드의 구조를 한 눈에 파악할 수 있도록 한다. 본 제안방식인 PDG 기반의 소스코드 유사도 검사 기법은 소스코드 dot 변환 단계, NetworkX 그래프 변환 단계, 그래프 유사도 검사 단계로 나뉜다. <그림 V-2-1>은 본 제안방식의 동작 과정을 간단히 나타낸 전체 시나리오이다.

1. 소스코드 dot 변환 단계

소스코드 dot 변환 단계는 PDG 기반의 소스코드 유사도 검사 기법을 진행하기 위해 소스코드를 PDG로 변환하기 위한 dot 파일을 생성하는 단계이다. 소스코드를 dot 파일로 변환하여 PDG로 표현할 경우 소스코드 내 제어 종속성과 데이터 종속성으로 인해 소스코드의 전체적인 흐름을 파악할 수 있다.

또한, 소스코드의 구조적 변경에도 강해 변수명 변경, 함수명 변경, 소스코드 순서 변경 등의 구조적 변경에 영향을 받지 않는다. 이는 PDG가 프로그램의 근본적인 논리적 종속성에 중점을 두기 때문이다. 또한, PDG는 정적 분석을 기반으로 하여 프로그램이 실행되지 않아도 유사도 검사를 진행할 수 있다. 이를 통해 기존 기법들보다 효율적인 소스코드 유사도 검사가 가능하다. 본 제안방식은 소스코드를 dot 파일로 변환하기 위해 파이썬 라이브러리인 Pyan3를 사용해 소스코드를 dot 파일로 변환했다. 소스코드 dot 변환 단계는 다음과 같다.

- Step1. 소스코드를 입력하여 PDG 생성을 위한 호출 그래프를 생성한다. 소스코드 내 모듈명 혹은 함수명 등을 저장하고 필요한 데이터 구조(정의 관계, 사용 관계 등)를 정의한다.
- Step2. 소스코드를 분석하여 소스코드 내 함수, 클래스 등의 정의를 분석

하고 호출 관계를 저장한다.

Step3. 생성된 호출 그래프를 기반으로 그래프 구조를 생성한다.

Step4. 그래프 구조, 그래프 옵션, 파일 이름을 기반으로 dot 파일 내 그래프 정보를 저장한다. 이는 이미지 형태의 그래프를 컴퓨터가 이해할 수 있는 언어로 변환하는 작업이다.

위와 같은 과정을 통해 소스코드를 그래프로 변환하고, 그래프로 변환된 소스코드를 컴퓨터가 이해할 수 있는 형식으로 변환한다. 이후 dot 파일로 변환된 두 소스코드를 PDG로 변환하여 그래프 유사도를 측정한다. 이를 통해 두 소스코드 간의 유사도를 측정할 수 있다.

알고리즘 3은 소스코드를 dot 파일로 변환하는 과정을 나타낸 알고리즘이다. 해당 알고리즘은 소스코드 파일 F 를 입력받아 소스코드 기반의 dot 파일을 생성한다. V 는 CallGraphVisitor를 통해 생성된 호출 그래프를 의미하며 G 는 호출 그래프를 기반으로 생성된 dot 파일의 그래프 구조 정보이다.

해당 알고리즘은 F 를 입력받고 이를 기반으로 호출 그래프 V 를 생성한다. 여기서 호출 그래프란 프로그램의 실행 흐름을 시각적으로 나타내는 그래프를 의미한다. 이후 생성된 호출 그래프를 기반으로 이를 dot 파일 생성 시 필요한 그래프 구조로 변환하여 G 를 생성한다. G 가 생성된 후, dot 파일 내에 그래프 구조를 작성하여 최종적으로 dot 파일을 생성한다. 생성된 dot 파일 내부에는 그래프의 노드 정의와 노드 간의 연결 관계가 정의되어 있다.

Algorithm 3 .dot File Transform

Input: Source code File F , Graph Options G_{opt}

Output: A .dot file

- 1: $filenames \leftarrow F.name$
 - 2: Set graph options G_{opt} based on input:
 - 3: $V \leftarrow CallGraphVisitor(filenames)$
 - 4: $G \leftarrow VisualGraph.from_visitor(V, G_{opt})$
 - 5: $writer \leftarrow DotWriter(G, G_{opt}, filename)$
 - 6: $writer.run()$
-

II. NetworkX 그래프 변환 단계

NetworkX 그래프 변환 단계는 dot 파일로 변환된 소스코드를 PDG로 변환 및 시각화하는 단계이다. NetworkX란 그래프 시각화를 위해 사용되는 Python 라이브러리이다. 그래프 유사도 검사 단계에서 그래프 편집 거리를 이용해 그래프 유사도 검사를 진행하기 위해선 그래프를 컴퓨터가 이해할 수 있는 형태로 변환해야 한다. NetworkX는 dot 파일로 저장된 소스코드를 컴퓨터가 이해할 수 있는 형태로 변환한다. 본 제안방식에선 이를 활용하여 dot 파일로 변환된 소스코드를 PDG로 변환한다.

NetworkX 그래프 변환 단계의 동작 과정은 다음과 같다.

Step1. dot 파일로 변환된 소스코드를 입력한다.

Step2. dot 파일 내 그래프 구조 정보를 불러온다.

Step3. 불러온 그래프 구조 정보를 NetworkX 호환 그래프인 pydot 형식으로 변환한다.

Step4. pydot 형태로 변환된 그래프를 변환하여 PDG를 생성한다.

위와 같은 과정을 통해 dot 파일 형식으로 저장된 그래프 정보를 최종적으로 그래프 유사도 검사에 필요한 PDG 형태로 변환할 수 있다. PDG로 변환된 그래프는 이후 그래프 편집 거리 알고리즘을 통해 소스코드 유사도 검사 시 활용된다.

알고리즘 4는 dot 파일을 PDG로 변환하는 과정을 설명한 알고리즘이다. 해당 알고리즘은 dot 파일 D 를 입력받아 PDG인 G_{nx} 를 출력한다. $read(D)$ 를 통해 D 에 저장된 그래프 구조 정보를 저장하고, 이후 D 를 이용해 NetworkX와 호환되는 dot 형식인 pydot 형식으로 변환한다. pydot 형식으로 변환된 그래프 구조 정보는 최종적으로 PDG 형태로 변환되어 G_{nx} 에 저장된다. 마지막으로 저장된 G_{nx} 을 반환하여 알고리즘을 종료한다.

Algorithm 4 .dot to Graph

Input: dot File D

Output: A NetworkX Graph G_{nx}

- 1: $data \leftarrow read(D)$
 - 2: $P_{list} \leftarrow dot_to_pydot(data)$
 - 3: $G_{nx} \leftarrow Convert_dot(P_{list})$
 - 4: **return** G_{nx}
-

III. 그래프 유사도 검사 단계

그래프 유사도 검사 단계는 위 두 과정을 통해 생성된 PDG를 기반으로 두 그래프 간의 구조적 유사도를 검사하는 단계이다. 두 PDG는 소스코드의 구조적 정보를 담고 있는 그래프로써 이를 통해 두 소스코드의 구조적 유사도를 효과적으로 측정할 수 있다. 또한, PDG의 종속성 정보를 통해 소스코드의 의미를 파악할 수 있어 이를 통해 소스코드의 의미까지 고려할 수 있다. 이러한 PDG를 기반으로 k -walk 기반의 휴리스틱 알고리즘을 적용한 그래프 편집 거리를 통해 유사도를 측정한다.

그래프 유사도 검사 단계의 동작 과정은 다음과 같다.

- Step1. 그래프 G 와 그래프 H 를 입력한다.
- Step2. 입력 받은 두 그래프에 대한 편집 연산 비용을 정의한다.
- Step3. 정의된 편집 연산 비용을 기반으로 비용 행렬을 생성하여 두 그래프 간의 최적의 비용을 갖는 편집 경로를 탐색한다.
- Step4. 만약 최적의 편집 비용을 갖는 편집 경로 계산 중 현재 비용이 최적의 비용을 초과할 경우 해당 탐색을 중지한다. 이는 탐색을 더 진행하여도 결과로 나온 비용이 최적의 비용이 아니므로 계산이 필요하지 않기 때문이다.
- Step5. 모든 편집 경로 계산 후 저장된 최적의 비용을 반환한다.

위 과정을 통해 두 그래프 간의 그래프 편집 거리를 계산할 수 있다. 알고리즘 5는 두 그래프와 탐색 거리 k 를 입력으로 하여 k -walk 기반의 휴리스틱 알고리즘을 적용한 그래프 편집 거리 알고리즘이다. 해당 알고리즘은 두 그래프의 최적의 근사값 $bestcost$ 를 계산하는 알고리즘이다. 그래프 입력 후

Algorithm 5 Graph Edit Distance

Input: Graphs G, H , Length of traversal k

Output: $bestcost$

```

1:  $bestcost \leftarrow \phi$ 
2: for each  $(path, cost)$  in  $Adv\_k\text{-walk\_Heuristic}(G, H, k)$  do
3:   if  $cost < bestcost$  or  $bestcost = None$  then
4:      $bestcost \leftarrow cost$ 
5:   end if
6: end for
7: return  $bestcost$ 

```

Algorithm 6 Adv_k-walk_Heuristic

Input: Graphs G, H , Length of traversal k

Output: $editpaths$

```

1:  $editpaths \leftarrow \emptyset$ 
2: Set  $cost_{ns}, cost_{nr}, cost_{ni}$  for nodes
3: Set  $cost_{es}, cost_{er}, cost_{ei}$  for edges
4: Set  $maxcost\_value$ 
5: for each node  $v_G$  in  $G$  do
6:    $B \leftarrow Adv\_GenWalk1(G, v_G, k)$ 
7:   for each node  $v_H$  in  $H$  do
8:      $B' \leftarrow Adv\_GenWalk1(H, v_H, k)$ 
9:      $cost \leftarrow CalculateCost(B, B', C_v, C_e)$ 
10:    if  $cost < maxcost\_value$  then
11:       $editpaths = (v_G, v_H, cost)$ 
12:    end if
13:  end for
14: end for
15: return  $editpaths$ 

```

$bestcost$ 를 초기화한다. 이후 $Adv_k\text{-walk_Heuristic}(G, H, k)$ 을 통해 두 그래프 간의 편집 비용의 근사값을 계산한다. 편집 비용 계산 후, 현재 계산된 $cost$ 가 $bestcost$ 보다 작을 경우, 현재 $cost$ 를 $bestcost$ 로 저장한다. 위 과정을 모든 그래프의 경로를 k 길이 만큼 탐색할 때까지 반복하여 최종적으로 그래프 G 를 H 로 변환하기 위해 필요한 최적의 편집 비용 근사값을 반환한다. 여기서 $Adv_k\text{-walk_Heuristic}(G, H, k)$ 는 알고리즘 6과 같다.

$Adv_k\text{-walk_Heuristic}(G, H, k)$ 는 두 그래프 간의 편집 경로를 계산하여 최적의 근사값을 계산하는 알고리즘이다. $Adv_GenWalk1$ 를 통해 그래프 G, H 의 k -walk 집합 B, B' 를 생성한다. 이후, B, B' 와 노드의 편집 비용 행렬 C_v , 엣지의 편집 비용 행렬 C_e 를 통해 그래프 G 를 H 로 변환하기 위한 편집 비용을 계산한다. 만약 계산된 편집 비용이 상한값을 초과하지 않을 경우, 현재 편집을 $editpaths$ 에 추가한다. 위 과정을 반복하여 최종적으로 그래프 G 를 H 로 변환하기 위한 최적의 근사값을 계산할 수 있다.

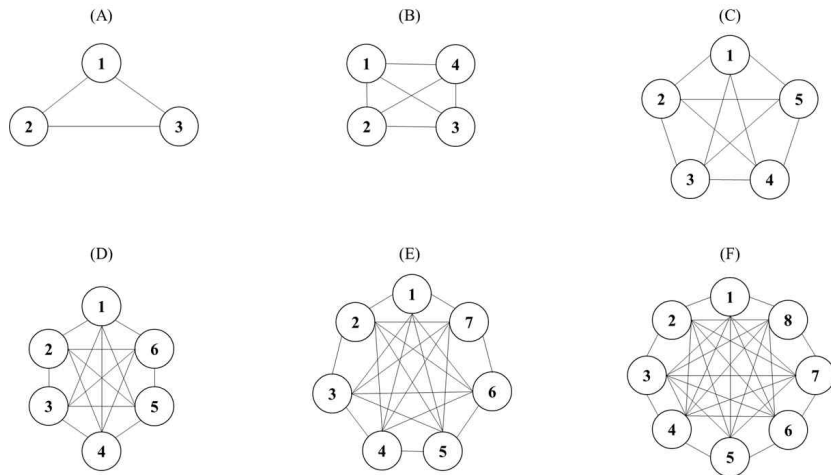
이러한 과정을 통해 PDG로 변환된 두 그래프를 그래프 편집 거리 알고리즘을 이용하여 두 그래프의 유사도를 계산할 수 있다. 그래프 편집 거리의 결과가 작을수록 두 그래프가 유사하다는 것을 의미하며, 이는 즉 두 소스코드가 구조적으로 유사한 소스코드임을 의미한다.

제 6장 제안방식 분석

본 장에선 본 제안방식이 3장 요구사항 분석 시 기술한 요구사항들을 만족하는지에 대하여 분석한다.

6.1 Tottering 현상을 방지하는 k -walk 생성 기법

본 절에서는 본 제안방식이 기존 k -walk 생성 기법을 개선함을 실험을 통해 설명한다. 실험은 기존 Gaüzère et al.의 k -walk 생성 기법과 비교하여 진행된다. 이는 본 제안방식이 Gaüzère et al.의 k -walk 생성 기법에서 발생한 Tottering 현상을 개선함을 보이기 위해 두 기법을 비교한다. 본 제안방식은 k -walk 생성 시 큐를 이용해 중복 방문을 관리하고, 이를 통해 Tottering 현상을 탐지하여 Tottering 현상이 발생한 k -walk 생성을 방지할 수 있다. 이를 증명하기 위해 본 제안방식과 기존 k -walk 생성 방식을 테스트를 진행하여 비교한다. 테스트에 사용된 그래프는 <그림 VI-1-1>과 같다.



<그림 VI-1-1> k -walk 생성 시 사용한 그래프

6.1.1 Tottering 현상 개선

기존 k -walk 생성 방식은 k -walk 생성 시 별도의 중복 방문을 탐지하지 않아 Tottering 현상이 발생한다. 이를 방지하기 위해 본 제안방식은 k -walk 생성 과정 중 큐를 사용해 중복 방문을 탐지한다.

해당 실험의 결과는 <그림 VI-1-1>과 같다. 해당 그림은 본 제안방식과 기존 k -walk 생성 기법을 통해 생성된 k -walk의 수를 나타낸 그래프이다. 실험 결과 본 제안방식으로 k -walk 생성 시 그래프 (A)에선 총 6개의 k -walk가 생성되었다. 더 나아가 그래프 (F)에선 총 2,612,736개의 k -walk가 생성되었다.

Gaüzère et al. 방식의 k -walk 생성 기법으로 k -walk 생성 시 그래프 (A)에선 총 12개의 k -walk가 생성되었다. 이는 본 제안방식과 총 6개의 k -walk 차이를 보이고 있다. 하지만 그래프 (F)에선 Gaüzère et al. 방식의 k -walk 생성 기법은 총 6,588,344개의 k -walk를 생성했다. 이는 본 제안방식과 총 3,975,608개의 차이를 보이고 있다. 따라서 본 제안방식은 기존 k -walk 생성 기법에서 발생하는 Tottering 현상을 문제를 해결했다.



<그림 VI-1-1> k -walk 생성 수

6.1.2 실행 시간 개선

기존 Gaüzère et al. 방식의 k -walk 생성 기법은 k -walk 생성 시 Tottering 현상이 발생해 그래프 구조 파악에 불필요한 k -walk가 생성되는 문제가 발생했다. Tottering 현상이 발생하여 그래프 구조 파악에 필요하지 않은 k -walk를 생성하며, 특히 탐색 초기에 Tottering 현상이 발생한 탐색 또한 k 번의 탐색을 끝마치기 전까지 탐색을 진행하기 때문에 k -walk 생성 시간 또한 크게 증가한다. 본 제안방식은 k -walk 생성 과정 중 중복 방문을 탐지하여 Tottering 현상을 방지하므로 k -walk 생성 시간 또한 크게 감소한다.

해당 실험의 결과는 <그림 VI-1-2-1>과 같다. 해당 그림은 본 제안방식과 기존 k -walk 생성 기법을 통해 k -walk 생성에 소요된 시간을 측정한 그래프이다. 실험 결과 본 제안방식으로 k -walk 생성 시 그래프 (A)에선 총 0.00003초가 소요됐다. 그래프 (F)에선 총 4.38023초가 측정되었다.

반면 Gaüzère et al. 방식의 k -walk 생성 기법은 그래프 (A)에서 총 0.00004초가 측정되었다. 이는 본 제안방식과 큰 차이가 아니지만 그래프 (F)에선 총



<그림 VI-1-2-1> k -walk 생성 시간

7.39398초로써 그래프가 커질수록 Gaüzère et al. 방식의 k -walk 생성 기법은 더욱 많은 시간이 소요된다.

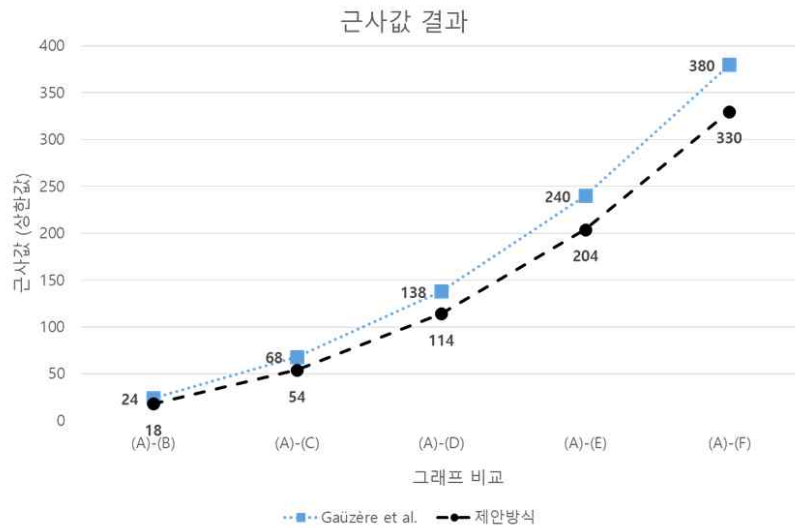
이 외 모든 실험에서도 Gaüzère et al. 방식의 k -walk 생성 기법과 비교해 본 제안방식은 모든 실험에서 보다 빠른 생성 시간을 보이고 있다.

6.1.3 정확도 개선

기존 Gaüzère et al. 방식의 k -walk 생성 기법은 불필요한 k -walk 생성으로 인해 다양한 문제가 발생한다. 위 두 문제와 더불어 LSAPE-GED의 근사값 계산 시 계산 결과의 정확도가 감소하는 문제가 발생한다. LSAPE-GED는 근사값의 상한, 즉 근사값의 최대값을 구하는 알고리즘으로써 이 상한이 작을수록 결과의 근사값이 기존 최적의 값과 유사함을 의미한다. 하지만 불필요한 k -walk로 인해 LSAPE-GED 계산에 필요한 비용 행렬 구축 시 비용 행렬의 크기가 커지는 문제점이 존재한다. 그로 인해 LSAPE-GED의 상한값 또한 커지게 되어 결과의 정확도에 영향을 미친다.

해당 실험의 결과는 <그림 VI-1-3-1>과 같다. 해당 그림은 본 제안방식과 기존 k -walk 생성 기법을 통해 생성된 k -walk를 기반으로 LSAPE-GED를 수행했을 때의 결과이다. 해당 실험은 그래프 (A)를 기준으로 각각 (B), (C), (D), (E), (F) 그래프를 $k=2$ 로 비교했을 때의 결과이다. 실험 결과 본 제안방식을 적용한 후 LSAPE-GED를 통해 (A)와 (B) 비교 시 근사값은 18이며 그래프 (A)와 (F) 비교 시 근사값은 330이다.

하지만 Gaüzère et al. 방식의 k -walk 생성 기법을 적용한 후 LSAPE-GED를 통해 (A)와 (B) 비교 시 근사값은 24로 본 제안방식을 적용했을 때 보다 높은 상한을 보였다. 또한, (A)와 (F) 비교 시 근사값은 380으로 이 또한 본 제안방식보다 높은 상한을 보였다. 그 외 실험에서도 Gaüzère et al. 방식의 k -walk 생성 기법을 적용한 결과는 모두 본 제안방식을 적용한 결과보다 높은 상한을 보였다. 따라서 본 제안방식은 Gaüzère et al. 방식의 k -walk 생성 기법보다 정확한 상한을 계산할 수 있다.



<그림 VI-1-3-1> 근사값 결과

6.2 PDG 기반의 소스코드 유사도 검사 알고리즘

본 절에서는 PDG 기반의 소스코드 유사도 검사 알고리즘의 요구사항을 본 제안방식이 만족함을 설명한다. 본 제안방식은 기존 텍스트 기반의 소스코드 유사도 검사의 단점을 개선하고, 이외 다른 그래프 기반의 소스코드 유사도 검사 기법의 한계점을 개선한 기법이다. 본 제안방식은 두 소스코드를 PDG로 변환하여 소스코드의 구조적 형태를 파악할 수 있고, 이 PDG를 이용해 두 소스코드 간의 구조적 유사도를 측정하는 기법이다. 본 제안방식이 3.2 절의 요구사항을 만족함을 이하 절에서 설명한다.

6.2.1 소스코드 기반의 PDG 생성

기존 텍스트 기반의 소스코드 유사도 검사는 소스코드 텍스트 변경, 구조적 변경에 매우 취약하다. 또한, 의미론 기반의 소스코드 유사도 검사는 소스코드를 바이너리 변환, CFG 변환, 의미론 그래프 변환과 같은 세 단계를 통해 유

사도 검사에 필요한 그래프를 생성한다. 또한, 정적 컴파일 언어에만 적용할 수 있으므로 특정 언어만 지원이 가능하다는 한계점이 존재한다.

추상 구문 트리 기반의 소스코드 유사도 검사는 구문적 유사도만 측정이 가능하며 소스코드의 의미적, 구조적 유사도는 측정이 불가능하다. 추가적으로, 소스코드의 의미를 고려할 수 없기에 구조적 변형에 취약하다.

소스코드 흐름도 기반의 소스코드 유사도 검사는 동적 분석을 기반으로 하기에 소스코드가 실행 불가능할 경우 유사도 검사가 불가능하며 구조적 변경이 발생해도 이를 탐지하기 어렵다.

PDG는 이러한 기존 텍스트 기반의 소스코드 유사도 검사 기법과 기존 그래프 기반의 소스코드 유사도 검사 기법의 한계를 극복할 수 있는 효과적인 그래프로써 본 제안방식은 이를 활용해 소스코드 유사도 검사 기법을 수행한다. 본 제안방식은 소스코드를 dot 파일로 변환하여 컴퓨터가 이해할 수 있는 형식으로 그래프를 변환하고, 해당 dot 파일을 기반으로 소스코드 유사도 검사에 활용할 수 있는 PDG를 생성한다. 이후 생성된 PDG를 기반으로 그래프 편집 거리를 활용해 소스코드 유사도 검사를 진행할 수 있다.

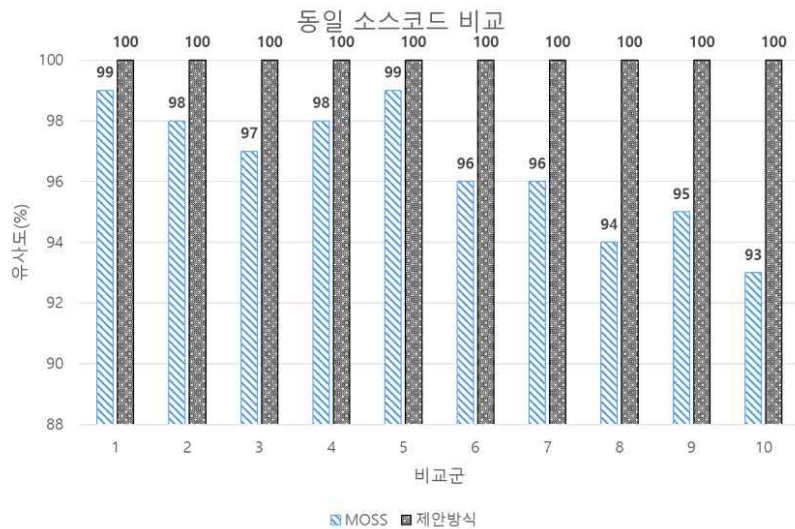
6.2.2 신뢰성

텍스트 기반의 소스코드 유사도 검사는 동일한 소스코드를 비교함에도 유사도 비교 결과가 98%, 97% 등 100%가 아닌 결과를 출력하는 사례가 존재한다. 이는 유사도 검사 기법의 신뢰도에 영향을 미칠 수 있으며, 유사도 탐지 시 오탐 또는 미탐 등이 발생할 확률을 포함하고 있다.

이를 해결하기 위해 본 제안방식은 PDG 기반의 소스코드 유사도 검사를 활용하여 소스코드의 구조적 유사도를 비교함으로써 동일한 소스코드를 완벽하게 식별할 수 있다. <그림 VI-2-2-1>은 본 제안방식과 MOSS를 이용해 동일한 소스코드들을 기반으로 유사도 검사를 실행했을 때의 결과이다.

실험 결과 MOSS는 10개의 실험군에서 동일한 소스코드를 이용해 비교했음에도 높게는 99%의 유사도를 보였다. 하지만 동일한 소스코드임에도 특정 실험군은 93%, 94% 등 상대적으로 낮은 유사도를 보였다.

반면 본 제안방식인 PDG 기반의 소스코드 유사도 검사는 동일한 소스코드



〈그림 VI-2-2-1〉 동일 소스코드 기반 유사도 검사 결과

를 비교했을 경우, 모든 실험군에서 100%의 유사도를 검출할 수 있다. 따라서 본 제안방식은 텍스트 기반의 소스코드 유사도 검사보다 높은 신뢰성을 보일 수 있다.

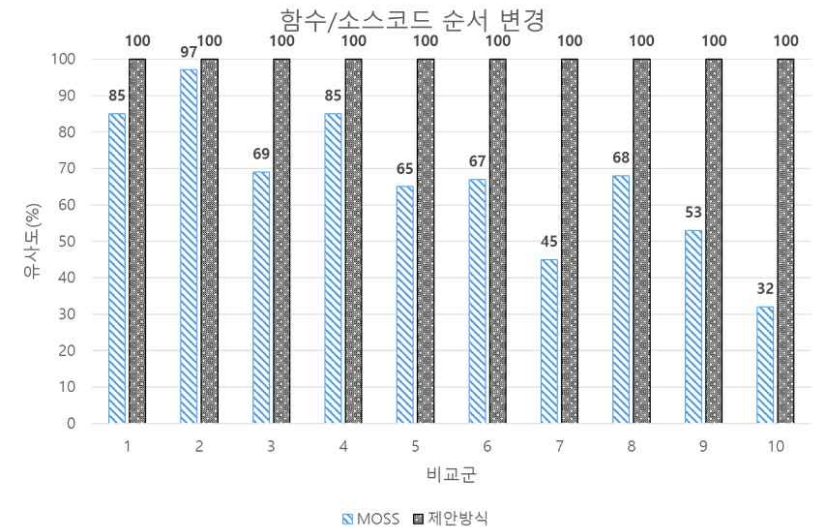
6.2.3 코드 변형 대응

대부분의 텍스트 기반의 소스코드 유사도 검사는 변수명 변경, 함수명 변경에 취약한 모습을 보인다. 하지만 MOSS의 경우 전처리 과정을 통해 변수명, 함수명 등을 제외하므로 이에 대한 내성을 갖고 있다. 하지만 MOSS는 소스코드의 순서를 바꾸거나 구조를 바꾸는 등의 수정을 할 경우, 유사도가 크게 감소하는 문제가 발생한다. 반면 본 제안방식은 소스코드의 구조적 변형에 큰 내성을 갖추고 있다.

〈그림 VI-2-3-1〉은 본 제안방식과 MOSS를 이용해 원본 소스코드와 소스코드의 순서를 변경한 소스코드를 비교했을 때의 결과이다. 해당 실험 또한 본

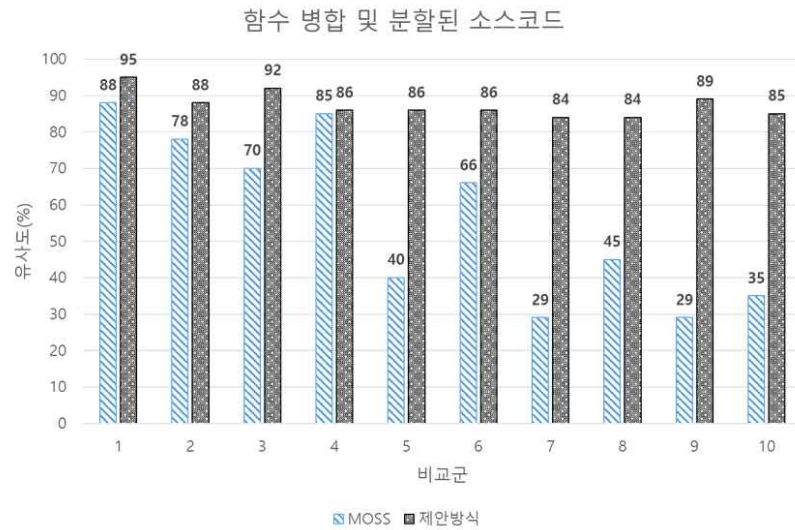
제안방식은 모든 실험군에서 100%의 유사도를 보였다. 하지만 MOSS의 경우 가장 높은 유사도를 보인 것은 97%이며 가장 낮은 유사도는 32%로써 매우 큰 차이가 난다. 즉, MOSS와 같은 텍스트 기반의 소스코드 유사도 검사는 함수 순서 변경, 소스코드 순서 변경 등에 큰 영향을 받아 정상적인 유사도 검사가 불가능하다.

〈그림 VI-2-3-2〉는 원본 소스코드와 소스코드 내 함수를 병합 또는 분할한 소스코드를 본 제안방식과 MOSS를 이용해 비교했을 때의 결과이다. 해당 실험에서 본 제안방식 또한 100%의 유사도를 보이지 못한다. 이는 소스코드 내 함수를 병합 및 분할하여 구조적 수정을 가했기 때문이다. 하지만 본 제안방식은 MOSS와 비교했을 때 상대적으로 높은 유사도를 보여 구조적 수정이 일어난 소스코드를 상대적으로 정확히 식별할 수 있다. 하지만 MOSS의 경우 함수 병합 및 분할이 심하게 일어날수록 최저 29%의 유사도를 보이고 있다. 따라서 본 제안방식은 텍스트 기반의 소스코드 유사도 검사에 비해 소스코드 변형에 강한 내성을 갖고 있다.



〈그림 VI-2-3-1〉 동일 소스코드 기반 유사도 검사 결과

제 7장 결론



〈그림 VI-2-3-2〉 동일 소스코드 기반 유사도 검사 결과

OSS를 활용하여 개발 시 OSS 혼용으로 인해 소스코드 공개, 저작권 위반으로 인한 법적 처벌 등의 피해가 발생할 수 있다. 이러한 문제를 방지하기 위해 소스코드 유사도 검사를 활용할 수 있다. 하지만 기존 텍스트 기반의 소스코드 유사도 검사는 소스코드 수정을 통한 유사도 검사 회피가 가능하다. 이를 방지하기 위해 그래프 기반의 소스코드 유사도 검사 기법을 활용할 수 있다. 그러나 기존 그래프 기반의 소스코드 유사도 검사는 특정 프로그래밍 언어 종속, 소스코드 의미 고려 불가, 높은 계산 비용 등의 문제점이 존재한다.

본 논문에선 이를 해결하기 위해 PDG 기반의 소스코드 유사도 검사 기법을 제안했다. PDG를 활용함으로써 다양한 프로그래밍 언어를 지원함과 동시에 소스코드의 의미를 고려할 수 있다. 하지만 PDG 기반의 소스코드 유사도 검사 또한 높은 계산 비용이 발생한다. 이를 해결하기 위한 방법 중 하나로 휴리스틱 알고리즘을 접목하는 것이다.

하지만 기존 k -walk 기반의 휴리스틱 알고리즘은 k -walk 생성 시 Tottering 현상으로 인해 효율성이 감소하는 문제점이 존재한다. 본 논문에선 이러한 Tottering 현상을 방지하는 새로운 k -walk 생성 기법을 제안함으로써 불필요한 k -walk생성을 방지한다. 이로 인해 소스코드 유사도 검사 시 더욱 효율적으로 유사도 검사가 가능하다.

본 제안방식은 유효한 k -walk만을 생성해 기존 k -walk 생성 기법보다 약 50% 적은 k -walk를 생성한다. 생성 시간 또한 본 제안방식이 그래프가 커질수록 최대 68% 정도 빠르게 측정됐다. 소스코드 유사도 검사 결과 또한 기존 MOSS와 비교해 구조적 변형에 강한 모습을 보였으며 본 제안방식이 10개의 실험에서 15~68% 높은 유사도 측정 결과를 보였다. 또한, 복잡한 소스코드 수정인 병합/분할에서도 본 제안방식이 MOSS보다 12~71% 높은 결과를 보였다.

이러한 본 제안방식을 교육 분야에 활용하여 소스코드 표절을 효과적으로 탐지할 뿐만 아니라 개발 환경에서 프로젝트 내 포함된 OSS를 효과적으로 탐지하는데 사용될 수 있다.

참고 문헌

- [1] O' Regan, G., & O' Regan, G. (2018). Open-source software. In *The Innovation in Computing Companion: A Compendium of Select, Pivotal Inventions* (pp. 209-212). Springer.
- [2] Segall, R. (2022). What is open source software (OSS) and what is big data?. In *Research Anthology on Big Data Analytics, Architectures, and Applications* (pp. 1-20). IGI Global.
- [3] Yanto, O. (2022). Legal consequences for perpetrators of violations of the economic rights of the creator and efforts to strengthen the protection of a song copyright work. *Jurnal IUS Kajian Hukum dan Keadilan*, 10(2), 185-195.
- [4] Putera, A., Sunardi, S., & Arief, H. (2023). Corporate responsibility for criminal actions of song copyright under the 28 law of 2014. *International Journal of Law, Environment, and Natural Resources*, 9(3), 47-56.
- [5] Abdillah, D., Agustina, L., Cipta, H., & Kejahatan, H. (2022). Copyright infringement crime in Islamic criminal law. *Legalite: Jurnal Perundang Undangan dan Hukum Pidana Islam*, 9(1), 113-122.
- [6] Kavanagh, P. (2004). *Examples of open source licenses*. MIT Press.
- [7] Kumar, A., Gupta, A., Sanagavarapu, L., & Reddy, Y. (2022). An approach to open-source software license management using blockchain-based smart-contracts. In *Proceedings of the 15th Innovations in Software Engineering Conference* (pp. 1-10). IEEE.
- [8] 공개SW포털. (n.d.). Retrieved October 12, 2024, from https://www.oss.kr/oss_guide/show/edcf990f-38e7-4918-8c91-d989babf3c09
- [9] Karnalim, O. S., & Chivers, W. (2020). Preprocessing for source code similarity detection in introductory programming. In *Proceedings of the 20th Koli Calling International Conference on Computing Education Research* (pp. 1-10). ACM.
- [10] Xylogiannopoulos, K. (2020). Identifying social networks of programmers using text mining for code similarity detection. In *Proceedings of the 2020 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)* (pp. 643-650). IEEE.
- [11] Hrkút, P., Ďuračík, M., Toth, Š., & Meško, M. (2023). Current trends in the search for similarities in source codes with an application in the field of plagiarism and clone detection. In *Proceedings of the 2023 33rd Conference of Open Innovations Association (FRUCT)* (pp. 77-84). IEEE.
- [12] Ragkhitwetsagul, C., Krinke, J., & Clark, D. (2018). A comparison of code similarity analysers. *Empirical Software Engineering*, 23(4), 2464-2519.
- [13] Zakeri-Nasrabadi, M., Parsa, S., Ramezani, M., Roy, C., & Ekhtiarzadeh, M. (2023). A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges. *Journal of Systems and Software*, 111796. <https://doi.org/10.1016/j.jss.2023.111796>
- [14] Pintér, Á., & Szénási, S. (2023). Longest common subsequence-based

source code similarity. In Proceedings of the 2023 IEEE 17th International Symposium on Applied Computational Intelligence and Informatics (SACI) (pp. 123-128). IEEE.

[15] Cheers, H., Lin, Y., & Smith, S. (2020). Detecting pervasive source code plagiarism through dynamic program behaviours. In Proceedings of the Twenty-Second Australasian Computing Education Conference (pp. 1-10). ACM.

[16] Huang, Q., Song, X., & Fang, G. (2020). Code plagiarism detection method based on code similarity and student behavior characteristics. In Proceedings of the 2020 IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA) (pp. 167-172). IEEE.

[17] Suttichaya, V., Eakvorachai, N., & Lurkraisit, T. (2022). Source code plagiarism detection based on abstract syntax tree fingerprints. In Proceedings of the 2022 17th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP) (pp. 1-6). IEEE.

[18] Pathik, B., & Sharma, M. (n.d.). Detection of syntax similarity of source code using a graph-based hybrid technique. Manuscript in preparation.

[19] Liu, Z., Zhang, Z., Ma, S., Liu, D., Zhang, J., Chen, C., Liu, S., Ahmed, M., & Xiang, Y. (2023). SemDiff: Binary similarity detection by diffing key-semantics graphs. arXiv Preprint, arXiv:2308.01463.

[20] Cheers, H., Lin, Y., & Smith, S. (2021). Academic source code plagiarism detection by measuring program behavioral similarity. IEEE Access, 9, 50391-50412.

[21] Ji, Y., Cui, L., & Huang, H. (2021). BugGraph: Differentiating source-binary code similarity with graph triplet-loss network. In Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security (pp. 1053-1062). ACM.

[22] Wang, Y., Ye, Y., Wu, Y., Zhang, W., Xue, Y., & Liu, Y. (2023). Comparison and evaluation of clone detection techniques with different code representations. In Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE) (pp. 332-344). IEEE.

[23] Gaüzère, B., Bougleux, S., Riesen, K., & Brun, L. (2014). Approximate graph edit distance guided by bipartite matching of bags of walks. In Proceedings of the Joint IAPR International Workshop, S+SSPR 2014 (pp. 73-82). Springer.

[24] Riesen, K., Ferrer, M., & Bunke, H. (2015). Approximate graph edit distance in quadratic time. IEEE/ACM Transactions on Computational Biology and Bioinformatics, 17(2), 483-494.

[25] Zhang, Z., Yan, H. H., & Zhang, X. W. (2016). Code similarity detection by program dependence graph. In Proceedings of the 2016 International Conference on Computer Engineering and Information Systems (pp. 255-261). Atlantis Press.

[26] Yu, D., Yang, Q., Chen, X., Chen, J., & Xu, Y. (2023). Graph-based code semantics learning for efficient semantic code clone detection. Information and Software Technology, 156, 107130.

[27] Mehrotra, N., Agarwal, N., Gupta, P., Anand, S., Lo, D., & Purandare, R. (2021). Modeling functional similarity in source code with graph-based siamese networks. *IEEE Transactions on Software Engineering*, 48(10), 3771-3789.

[28] Hu, Y., Wang, H., Zhang, Y., Li, B., & Gu, D. (2019). A semantics-based hybrid approach on binary code similarity comparison. *IEEE Transactions on Software Engineering*, 47(6), 1241-1258.

[29] Haque, S., Eberhart, Z., Bansal, A., & McMillan, C. (2022). Semantic similarity metrics for evaluating source code summarization. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension* (pp. 36-47). IEEE.

[30] Zheng, M., Pan, X., & Lillis, D. (2018). CodEX: Source code plagiarism detection based on abstract syntax trees. In *Proceedings of the 26th AIAI Irish Conference on Artificial Intelligence and Cognitive Science* (pp. 1-10). CEUR Workshop Proceedings.

[31] Svacina, J., Simmons, J., & Cerný, T. (2020). Semantic code clone detection for enterprise applications. In *Proceedings of the 35th Annual ACM Symposium on Applied Computing* (pp. 1-6). ACM.

[32] Zhang, L., Liu, D., Li, Y., & Zhong, M. (2012). AST-based plagiarism detection method. In Wang Y., & Zhang X. (Eds.), *Internet of Things. Communications in Computer and Information Science*, Vol. 312. Springer.

[33] Zhang, F., Li, L., Liu, C., & Zeng, Q. (2020). Flow chart generation-based source code similarity detection using process mining.

Scientific Programming, 2020, 8865413:1-8865413:15.

[34] Ferrante, J., Ottenstein, K., & Warren, J. (1987). The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems*, 9(3), 319-349.

[35] Krinke, J. (2001). Identifying similar code with program dependence graphs. In *Proceedings of the 8th Working Conference on Reverse Engineering* (pp. 301-309). IEEE.

[36] Gao, X., Xiao, B., Tao, D., & Li, X. (2010). A survey of graph edit distance. *Pattern Analysis and Applications*, 13, 113-129.

[37] Brun, L., & Foggia, P., & Vento, M. (2018). Trends in graph-based representations for pattern recognition. *Pattern Recognition Letters*, 113(4), 23-35.

[38] Conte, D., Foggia, P., Sansone, C., & Vento, M. (2004). Thirty years of graph matching in pattern recognition. *International Journal of Pattern Recognition and Artificial Intelligence*, 18(3), 265-298.

[39] Foggia, P., Percannella, G., & Vento, M. (2014). Graph matching and learning in pattern recognition in the last 10 years. *International Journal of Pattern Recognition and Artificial Intelligence*, 28(1), 1450001:1-1450001:40.

[40] Zeng, Z., Tung, A., Wang, J., Feng, J., & Zhou, L. (2009). Comparing stars: On approximating graph edit distance. *Proceedings of the VLDB Endowment*, 2(1), 25-36.

[41] Justice, D., & Hero, A. (2006). A binary linear programming formulation of the graph edit distance. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(8), 1200-1214.

[42] Blumenthal, D. B., Boria, N., Gamper, J., Bougleux, S., & Brun, L. (2020). Comparing heuristics for graph edit distance computation. *The VLDB Journal*, 29(1), 419-458.

[43] Bougleux, S., Gaüzère, B., Blumenthal, D. B., & Brun, L. (2020). Fast linear sum assignment with error-correction and no cost constraints. *Pattern Recognition Letters*, 134, 37-48.

[44] Luo, L., Ming, J., Wu, D., Liu, P., & Zhu, S. (2014). Semantics-based obfuscation-resilient binary code similarity comparison with applications to software plagiarism detection. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (pp. 389-400). ACM.

ABSTRACT

A Study on PDG-based Source Code Similarity Checking and k -walk Generation for Efficient OSS Similarity Checking

Yoon, Seongcheol

Department of Software Convergence
Soonchunhyang University
Asan, Korea

(Supervised by Professor Kim, Su-Hyun)

Modern development environments actively utilize OSS to reduce development time and cost. When using OSS, you must comply with the OSS license, which governs its use. However, OSS licenses may not be compatible with each other, as different licenses have different rules. OSS is strictly intellectual property protected by copyright, and violating an OSS license can result in penalties such as copyright infringement. However, due to the collaborative nature of development environments, it is very difficult to identify the OSS used by other developers. This can lead to problems such as withdrawing commercial software from the market or having to disclose the full source code. In this paper, we propose a novel source code similarity checking technique for identifying OSS in such software. The proposed method converts the source code into a graph and measures the

similarity by comparing the structural similarity of the source code. In this case, it effectively prevents similarity detection evasion such as syntax changes and structural changes that occurred in existing text-based source code similarity checks and enables more accurate similarity checks. However, additional computational cost is incurred when checking source code similarity after graph conversion, which reduces efficiency. To solve this problem, we propose a new k-walk-based graph heuristic algorithm that improves the tottering phenomenon that occurred in the existing k-walk-based graph heuristic algorithm. It is expected to be actively used in development environments that require OSS identification and educational environments that require source code similarity checks, as it can identify OSS in software more accurately and efficiently.

Keyword

Open Source Software, Source Code Similarity, Graph, Program Dependence Graph, Graph Similarity, Heuristic algorithms